

Bí mật

Ngoài ph

BSP cho S800
PS-CPU
Đặc tả điều khiển chi tiết

Ký hiệu phiên bản	Ngày	Hạng mục	Người phê duy ệt	Người rà soát	Người soạn thả o	Bộ phận phát hành	Mã đăng ký
A	2017/3/22	Bản đầ u tiên			Hirota	Trụ sở Phát triển Kinh doanh Thiết bị Thông tin Trung tâm Phát triển Đơn vị Công nghệ Điện tử Phòng Phát triển Module	S800-BSP_DD-27(01)
B	2017/6/23	Bản B			Hirota Miwa	Trụ sở Phát triển Thiết bị Thông tin Trung tâm Phát triển Nền tảng H ệ thống Phòng Phát triển Nền tảng số 2	S800-BSP_DD-27(02)
C	2017/09/13	Bản C			Watana be	Trụ sở Phát triển Thiết bị Thông tin Trung tâm Phát triển Nền tảng H ệ thống Phòng Phát triển Nền tảng số 2	S800-BSP_DD-27(03)

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

Lịch sử sửa đổi

Ký hiệu phiên bản	Ngày	Nội dung sửa đổi	Lý do sửa đổi	Người phụ trách
A	2016/07/22	Bản đầu tiên	Tạo mới để hỗ trợ đối tượng S800	Hirota
B	2016/12/20	■ Xử lý vấn đề CT Bổ sung thông tin còn thiếu khi tạo đặc tả CT Thay đổi đặc tả thanh ghi I2C Thay đổi do dung lượng ROM của phần Main không đủ Xử lý lỗi hoạt động của RTC Sửa lỗi	5.1.4.1. Đặc tả truyền thông 5.1.4.2. Thiết lập STM32CubeMX 5.1.6. Điều khiển RTC 5.1.6.1. Thiết lập STM32CubeMX 5.1.5.3.3 Mở rộng I/O 5.1.5.3. Phân vùng Flash ROM 5.3. Danh sách hàm 5.4. Đặc tả hằng số 5.6. Đặc tả cấu trúc (struct) 5.9. Đặc tả hàm	Miwa
B	2017/06/23	■ Thay đổi tương ứng với việc sửa đổi tài liệu đặc tả phần cứng Thay đổi tên tài liệu đặc tả được tham chiếu Hiệu chỉnh do thay đổi phần cứng Bổ sung phần mô tả	1.4. Tài liệu liên quan Toàn bộ các chương liên quan đến điều khiển tín hiệu 5.2. Sơ đồ tuần tự tổng quan 5.2.1. Sơ đồ tuần tự tổng thể 5.2.2. Sơ đồ tuần tự bật nguồn (Power ON) 5.2.3. Sơ đồ tuần tự tắt nguồn (Power OFF)	Hirota
C	2017/09/13	■ Hỗ trợ bổ sung PS-CPU - Xử lý mất điện tức thời khi bật nguồn (Power ON) - Hỗ trợ ghi lại (rewrite) chương trình PS-CPU - Hỗ trợ checksum / thông tin phiên bản	4.2 Danh sách source/thư mục 5.1.5.3.3 Mở rộng I/O Checksum / phiên bản 5.1.5.5 Điều khiển reset khi ghi lại chương trình 5.1.7 Xử lý mất điện tức thời khi bật nguồn lần đầu 5.1.8 Checksum / thông tin phiên bản 5.3. Danh sách hàm 5.4. Danh sách hằng số 5.7. Biến toàn cục 5.9.1.10 power_monitor_proc 5.9.1.13 s800_power_off 5.9.1.16 ap_power_en_proc 5.9.1.28 checksum_calc 5.9.2.1. main 5.9.2.9. msw_on_proc_iap 5.9.2.10 km_exti_callback_iap 5.9.2.11 km_it_init_iap	Watanabe

《Mục lục》

1. Giới thiệu	8
1.1. Mục đích	8
1.2. Phạm vi áp dụng	8
1.3. Tổng quan	8
1.4. Tài liệu liên quan	8
2. Định nghĩa thuật ngữ	9
3. Đặc tả phần cứng	10
3.1. PS-CPU Thông tin cơ bản	10
3.1.1. Tổng quan	10
3.1.2. Bản đồ bộ nhớ (Memory Map)	10
3.2. Sơ đồ cấu hình phần cứng	11
3.2.1. S800 Sơ đồ nền tảng	11
3.2.2. Mạch kết nối	12
3.2.3. Mạch nguồn	13
3.2.1. Gán chân (Pin Assignment)	14
3.3. Đặc tả thanh ghi	15
4. Cấu hình phần mềm	16
4.1. Sơ đồ cấu hình phần mềm	16
4.2. Danh sách source/thư mục	16
4.3. Môi trường phát triển	18
4.3.1. STM32CubeMX	18
4.3.1.1. Ngoại vi (Peripheral), PIN Phương pháp cấu hình	18
4.3.1.2. Sinh mã (Code Generation)	19
4.3.2. µVision	20
4.3.2.1. Multi-Project (nhiều dự án)	20
4.3.2.2. Build (biên dịch)	20
4.3.2.3. Trình gỡ lỗi (Debugger)	21
4.3.2.4. Thêm file	21
5. Đặc tả điều khiển	22
5.1. SWTổng quan điều khiển	22
5.1.1. KMTín hiệu đầu vào	22
5.1.1.1. MONI_24V11	22
5.1.1.2. SB_PG	22
5.1.1.3. MC3.3VON_MONI	22
5.1.2. KMTín hiệu đầu ra	23
5.1.2.1. IR_P_ON	23
5.1.2.2. ERP_SENSOR_ON	23
5.1.2.3. /RESET	23
5.1.2.4. SB_PWR_EN	23
5.1.2.5. MC_P_ON	23
5.1.2.6. /DISCHG	23
5.1.2.7. /RST_SLP2	23
5.1.2.8. /USB2_OE	24
5.1.2.9. MC_PWR_EN	24
5.1.2.10. WAKEUP	24
5.1.3. KMTín hiệu ngắt	24
5.1.3.1. /HRESET_REQ	24
5.1.3.2. AP_PWR_EN	24
5.1.3.3. POWER_MONITOR	24
5.1.3.4. MSW_ON	24
5.1.4. I2CĐiều khiển	25
5.1.4.1. Đặc tả truyền thông	25
5.1.4.2. STM32CubeMXThiết lập	25
5.1.4.3. Trình tự điều khiển	27
5.1.4.4. PS-CPU 2 nd Đặc tả địa chỉ	28
5.1.5. IAPĐiều khiển	34

5.1.5.1. IAPCơ sở của điều khiển	34
5.1.5.2. Phân tách dự án	34
5.1.5.3. FlashROMPhân vùng	35
5.1.5.4. I2CTruyền thông	35
5.1.5.5. Điều khiển reset khi ghi lại chương trình	36
5.1.6. RTCĐiều khiển	37
5.1.6.1. STM32CubeMXThiết lập	37
5.1.7. Bật nguồn lần đầu (Power ON) – Xử lý mất điện tức thời	38
5.1.7.1. Bật nguồn lần đầu (Power ON) – Trình tự	38
5.1.8. Checksum / Thông tin phiên bản	39
5.2. Sơ đồ tuần tự tổng quan	40
5.2.1. Sơ đồ tuần tự tổng thể	40
5.2.2. Sơ đồ tuần tự nguồn (Power ON)	41
5.2.3. Sơ đồ tuần tự nguồn (Power OFF)	42
5.3. Danh sách hàm	43
5.3.1. main project	43
5.3.2. iap project	45
5.4. Đặc tả hằng số	45
5.4.1. main project	45
5.4.2. iap project	49
5.5. Đặc tả kiểu liệt kê (enum)	51
5.5.1. main project	51
5.5.2. iap project	51
5.6. Đặc tả cấu trúc (struct)	51
5.6.1. main project	52
5.6.1. iap project	52
5.7. Biến toàn cục	52
5.7.1. main project	52
5.7.2. iap project	53
5.8. const (danh sách định nghĩa)	53
5.8.1. main project	53
5.9. Đặc tả hàm	54
5.9.1. main project	54
5.9.1.1. main	54
5.9.1.2. jump2iap	54
5.9.1.3. Delnit	55
5.9.1.4. change_cmd2array	55
5.9.1.5. io_extend_read	56
5.9.1.6. io_extend_write	56
5.9.1.7. io_extend_memory_write	57
5.9.1.8. internal_sts_init	57
5.9.1.9. mc3_3von_moni_proc	58
5.9.1.10. power_monitor_proc	58
5.9.1.11. hreset_req_proc	59
5.9.1.12. extend_output_init	59
5.9.1.13. s800_power_off	59
5.9.1.14. moni_24v_proc	60
5.9.1.15. s800_power_on	60
5.9.1.16. ap_power_en_proc	61
5.9.1.17. erp_sensor_proc	61
5.9.1.18. rst_usb_proc	61
5.9.1.19. mc_p_on_proc	62
5.9.1.20. ir_p_on_proc	63
5.9.1.21. mc_oe_proc	63
5.9.1.22. mc_pwr_en_proc	64
5.9.1.23. msw_on_proc	64
5.9.1.24. rx_comp	65
5.9.1.25. change_state	65

5.9.1.26. i2c_recv_first	66
5.9.1.27. i2c_recv_wait	67
5.9.1.28. checksum_calc	68
5.9.1.29. hreset_req_proc_handler	68
5.9.1.30. km_exti_callback	68
5.9.1.31. km_it_init	69
5.9.1.32. HAL_TIM_PeriodElapsedCallback	69
5.9.1.33. km_timer_set	70
5.9.1.34. km_timer_get_state	70
5.9.1.35. adc_moni_5v_start_proc	71
5.9.1.36. adc_moni_5v_off_proc	71
5.9.1.37. BSP_PWR_enter_stopmode	71
5.9.1.38. BSP_PWR_enter_sleepmode	72
5.9.2. IAP project	72
5.9.2.1. main	72
5.9.2.2. rx_comp	73
5.9.2.3. change_state	73
5.9.2.4. i2c_recv_first	73
5.9.2.5. i2c_recv_wait	74
5.9.2.6. flash_write_32	74
5.9.2.7. main_project_flash_erase	75
5.9.2.8. check_sum_calc	75
5.9.2.9. msw_on_proc_iap	76
5.9.2.10. km_exti_callback_iap	76
5.9.2.11. km_it_init_iap	77
6. Kiểm chứng	78
6.1. Môi trường thực thi	78
6.2. Các quan điểm kiểm chứng	78
6.2.1. UTThực hiện kiểm chứng	78
6.2.2. Khởi động	78
6.2.3. Kiểm tra các loại ngắt	78
6.2.4. I2CTruyền thông	78
6.2.5. Điều khiển port	78
6.2.6. IAP	78
6.2.7. Xử lý mất điện tức thời	78

1. Giới thiệu

1.1. Mục đích

Tài liệu đặc tả này trình bày điều khiển chi tiết của PS-CPU trong BSP cho S800.

1.2. Phạm vi áp dụng

Tài liệu đặc tả này được áp dụng cho giai đoạn phát triển BSP cho S800.

1.3. Tổng quan

Tài liệu này mô tả cấu hình phần cứng của PS-CPU và điều khiển chi tiết của các chức năng sau đây của PS-CPU.

- Thiết lập môi trường, quy trình build
- Điều khiển IAP
- Điều khiển IO mở rộng
- Điều khiển các ngoại vi khác nhau

PS-CPU tương ứng với power SubCPU của các model trước đây (S72/73), nhưng do thay đổi nền tảng nên các chức năng cũng đã thay đổi rất nhiều.

Do các thay đổi về chức năng so với model trước đây (S72/73), nó được sử dụng như phần mở rộng của một phần điều khiển nguồn và I/O của nền tảng S800, vì vậy tên gọi được đổi từ power SubCPU thành PS-CPU.

Các điểm khác biệt chính so với model trước đây (S72/73) như sau

1. Không thực hiện điều khiển nguồn chi tiết
Việc điều khiển nguồn của các thiết bị về cơ bản do LPPP thực hiện, còn PS-CPU chủ yếu điều khiển như phần mở rộng của I/O.
2. Không thực hiện điều khiển mất điện tức thời
Điều khiển reset được thực hiện theo chỉ thị từ phía S800, PS-CPU không tự phán đoán để thực hiện điều khiển reset.
3. Không lưu dữ liệu vào vùng nhớ không mất dữ liệu (non-volatile)
Điều khiển tiết kiệm điện về cơ bản do LPPP thực hiện, phía PS-CPU có rất ít điều khiển nên không cần lưu tiến trình của chuỗi xử lý vào vùng non-volatile.
4. Không triển khai cổng debug (debug terminal)
Có thể sẽ được bổ sung tùy theo nhân lực, nhưng vì bản thân UART chỉ dùng cho debug nên sẽ được hoãn lại.

1.4. Tài liệu liên quan

No.	Tên file	Nội dung
1	¥¥10.128.16.110¥data1¥29P.System Integrated SoC Development¥03_BSP¥11.Reference Materials¥04.PS-CPU STM32F0x1_2_8RM(RM0091)_Japanese_rev1.pdf	Datasheet của STM32Fx1 (bản tiếng Nhật)
2	¥¥10.128.16.110¥data1¥29P.System Integrated SoC Development¥02_Board¥24_Power SubCPU control specification PS-CPU control specification(X).docx	Đặc tả điều khiển PS-CPU do nhóm HW soạn
3	¥¥10.128.19.213¥guest¥29P.System Integration SoC Development¥PS-CPU¥Doc¥Function Specifications Function Specifications.docx	Đặc tả chức năng của driver do Mikawa BSP soạn
4		
5		
6		
7		
8		

2. Định nghĩa thuật ngữ

Định nghĩa các thuật ngữ đặc biệt được sử dụng trong tài liệu đặc tả này được trình bày trong bảng dưới đây.

Thuật ngữ	Nội dung
STM32F031C6T6	Vi điều khiển của STmicro. Vi điều khiển hiệu năng cao, tiêu thụ điện năng thấp.
PS-CPU	CPU được gắn trên S800, điều khiển một phần nguồn điện và I/O mở rộng. RTC tích hợp trong CPU này được sử dụng với cách điều khiển giống RTC truyền thống. Viết tắt của Power Sub CPU, chỉ CPU Cortex-M0 Kiến trúc: v6-M, độ rộng bit của CPU: 32bit, số lõi: 1
AP	Viết tắt của Application Processor Khối chứa AP CM3/AP CA72
AON	Viết tắt của Always On Power Island Khối chứa LPPP/PMU, luôn được cấp nguồn ở mọi trạng thái khác "Power OFF"
PMU	Viết tắt của Power Management Unit, chỉ CPU Cortex-M3 nằm trong khối AON Kiến trúc: v7-M, độ rộng bit của CPU: 32bit, số lõi: 1 Thực hiện điều khiển nguồn theo chỉ thị từ LPPP. Khác với PS-CPU.
IAP	in-application programming viết tắt. IAP là chức năng ghi lại bộ nhớ flash của vi điều khiển trong khi chương trình người dùng đang chạy
CA72(CA72_0-3)	CPU Cortex-A72 nằm trong khối AP Kiến trúc: v8-A, độ rộng bit của CPU: 64bit, số lõi: 4 (QuadCore)
LPPP	Viết tắt của Low Power Packet Processor, CPU Cortex-R4 nằm trong khối AON Kiến trúc: v7-R, độ rộng bit của CPU: 32bit, số lõi: 1
STM32CubeMX	STM32CubeMX là công cụ sinh mã nguồn do STmicro cung cấp. Công cụ cho phép cấu hình PIN và bật/tắt các ngoại vi một cách dễ dàng bằng GUI.
µVision	µVision là môi trường phát triển tích hợp do ARM (Keil) cung cấp. Hỗ trợ trình biên dịch, quản lý mã nguồn, ulink v.v. và các trình gỡ lỗi sử dụng chúng.

3. Đặc tả phần cứng

3.1. Thông tin cơ bản về PS-CPU

3.1.1. Tổng quan

Tổng quan về PS-CPU như sau.

Mã sản phẩm	STM32F031C6T6
Core	Cortex-M0
CPU CLOCK	48MHz
Số lượng chân	48 chân
RAM	4KB
FLASH ROM	32KB
SPI	1 cổng
I2C	1 cổng
ADC	13 kênh

3.1.2. Bản đồ bộ nhớ (Memory map)

Về bản đồ bộ nhớ của PS-CPU, hãy tham khảo datasheet trong tài liệu tham chiếu.

Tham khảo các mục sau

2.2 Cấu hình bộ nhớ

2.2.1 Tổng quan

2.2.2 Bản đồ bộ nhớ và địa chỉ biên của thanh ghi

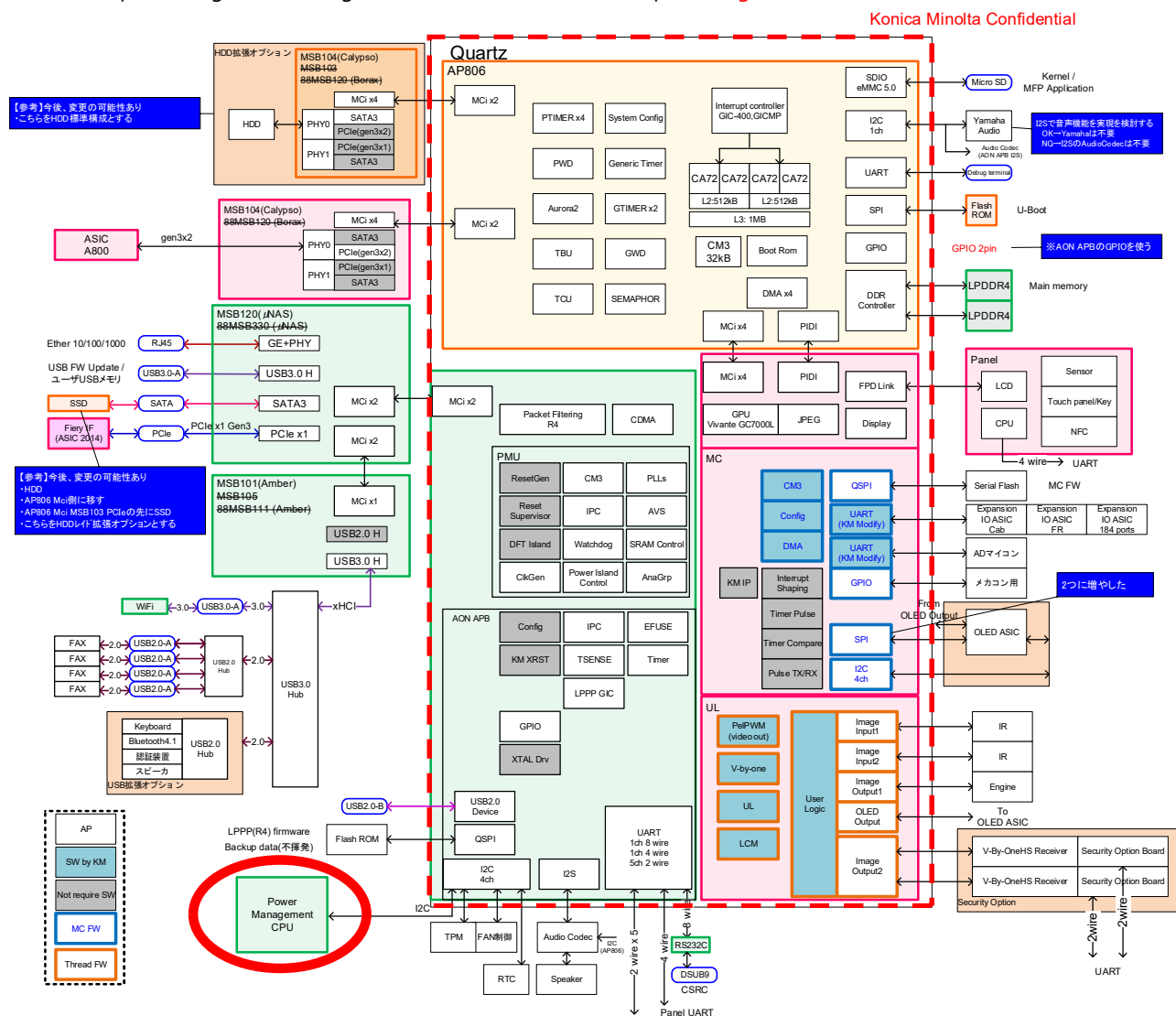
[Tài liệu tham khảo]

Tài liệu tham khảo 1: STM32F0x1_2_8RM(RM0091)_Japanese_rev1.pdf

3.2. Sơ đồ cấu hình phần cứng

3.2.1. Sơ đồ nền tảng S800

Sơ đồ khối phần cứng của nền tảng S800 như sau. PS-CPU nằm ở phần **vòng tròn đỏ**.



PS-CPU chủ yếu điều khiển việc cấp nguồn cho các linh kiện ngoại vi hoạt động phối hợp với khối SB (South Bridge) và khối AON (Always ON) của S800.

Việc điều khiển nguồn chi tiết của khối AP và khối AON do PMU đảm nhiệm, không phải do PS-CPU.

CPU chính (CA72), LPPP (R4) và PS-CPU giao tiếp với nhau qua I2C. Do đặc tính của truyền thông I2C, CPU chính và LPPP là thiết bị master, còn PS-CPU là thiết bị slave.

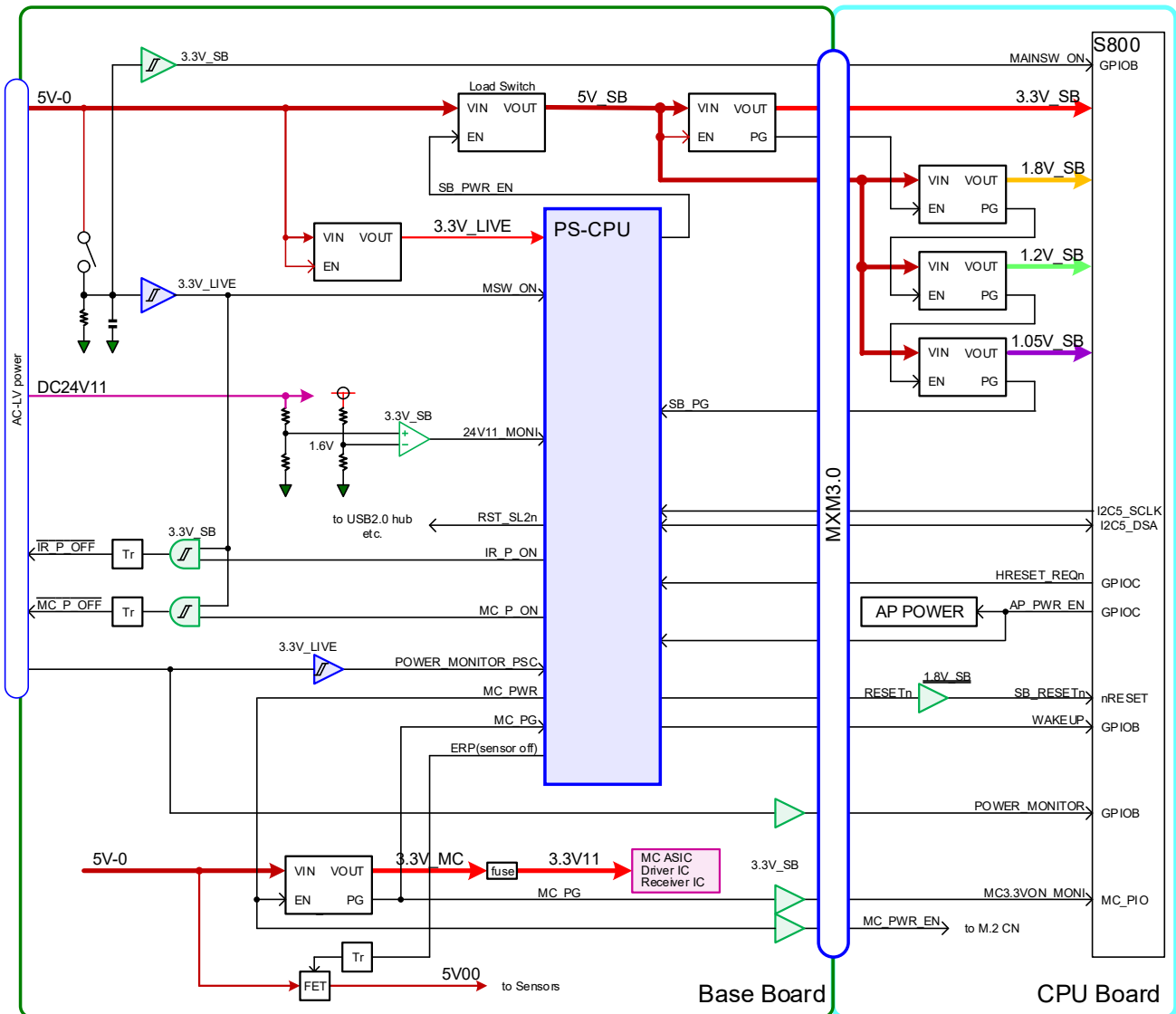
[Tài liệu tham khảo]

Tài liệu liên quan 2: PS-CPU control specification(X).docx

3.2.2. Mạch kết nối

Mô tả cấu hình phần cứng (mạch kết nối) của PS-CPU.

Cấu hình cơ bản như sơ đồ dưới đây.

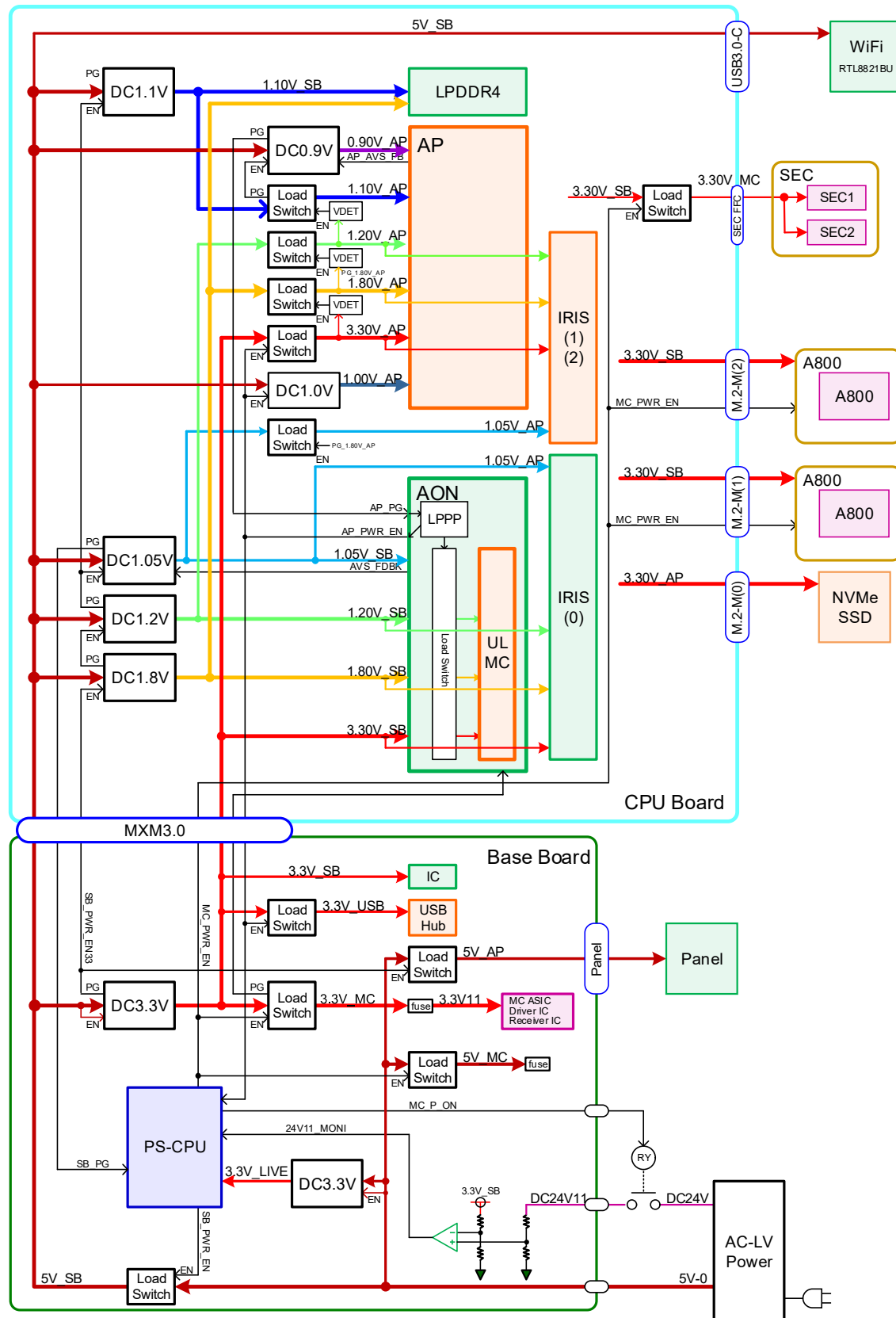


[Tài liệu tham khảo]

Tài liệu liên quan 2: PS-CPU control specification(X).docx

3.2.3. Mạch nguồn

Sơ đồ mạch nguồn như sau.

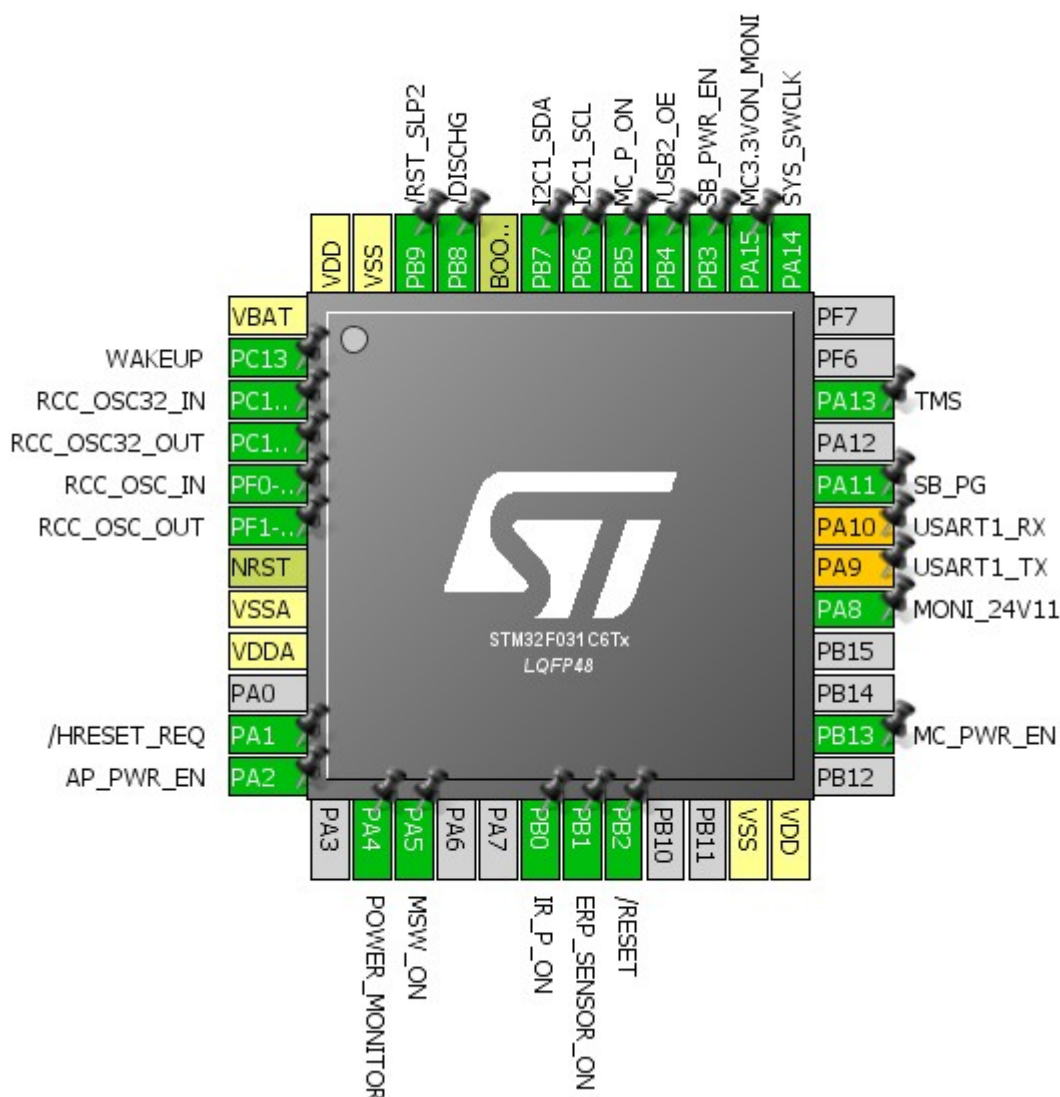


[Tài liệu tham khảo]

Tài liệu liên quan 2: PS-CPU control specification(X).docx

3. 2. 1. Gán chân (Pin assignment)

Việc gán chân cho S800 như sau.



Số chân	Loại chân	Chức năng	Tên tín hiệu	Ghi chú
2	PC13	GPIO_Output	WAKEUP	5.2.2.9.WAKEUP
3	PC14-OSC32_IN	RCC_OSC32_IN		
4	PC15-OSC32_OUT	RCC_OSC32_OUT		
5	PF0-OSC_IN	RCC_OSC_IN		
6	PF1-OSC_OUT	RCC_OSC_OUT		
11	PA1	GPIO_EXTI1	/HRESET_REQ	5.2.4.1./HRESET_REQ
12	PA2	GPIO_EXTI2	AP_PWR_EN	5.2.4.2.AP_PWR_EN
14	PA4	GPIO_EXTI4	POWER_MONITOR	5.2.4.3.POWER_MONITOR
15	PA5	GPIO_EXTI5	MSW_ON	5.2.3.4.MSW_ON
18	PB0	GPIO_Output	IR_P_ON	5.2.2.1.IR_P_ON
19	PB1	GPIO_Output	ERP_SENSOR_ON	5.2.2.3.ERP_SENSOR_ON
20	PB2	GPIO_Output	/RESET	5.2.2.4./RESET
21	PB10	GPIO_Output	/USB2_OE	5.2.2.8./USB2_OE
26	PB13	GPIO_Output	MC_PWR_EN	5.2.2.7.MC_PWR_EN
29	PA8	GPIO_Input	MONI_24V11	5.2.1.2.MONI_24V11

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

30	PA9*	USART1_TX		
31	PA10*	USART1_RX		
32	PA11	GPIO_Input	SB_PG	5.2.1.2.SB_PG
34	PA13	SYS_SWDIO	TMS	
37	PA14	SYS_SWCLK		
38	PA15	GPIO_Input	MC3.3VON_MONI	5.2.1.3.MC3.3VON_MONI
39	PB3	GPIO_Output	SB_PWR_EN	5.2.2.8.SB_PWR_EN
41	PB5	GPIO_Output	MC_P_ON	5.2.2.2.MC_P_ON
42	PB6	I2C1_SCL		
43	PB7	I2C1_SDA		
45	PB8	GPIO_Output	/DISCHG	5.2.2.6./DISCHG
46	PB9	GPIO_Output	/RST_SLP2	5.2.2.11./RST_USB

3.3. Đặc tả thanh ghi

Về đặc tả thanh ghi của PS-CPU, hãy tham khảo datasheet trong tài liệu tham chiếu.
 Vì đặc tả thanh ghi được cung cấp theo từng ngoại vi, hãy tham khảo các tài liệu đó.

[Tài liệu tham khảo]

Tài liệu tham khảo 1: STM32F0x1_2_8RM(RM0091)_Japanese_rev1.pdf

4. Cấu hình phần mềm

4.1. Sơ đồ cấu hình phần mềm

PS-CPU được cấu trúc thành hai project để có thể thực hiện IAP.

Main project (project chính)	Project chính của PS-CPU. Thực hiện điều khiển nguồn và điều khiển IO mở rộng.
IAP project	Project dùng để ghi lại (rewrite) Main project. Chương trình chạy khi nhận được lệnh ghi lại chương trình.

[Tài liệu tham khảo]

4.2. Danh sách source / thư mục

Cấu trúc thư mục của PS-CPU như sau.

common	Wrapper cho các driver không phụ thuộc vào CPU.
iap	Project cho xử lý IAP của PS-CPU
main	Project cho xử lý chính của PS-CPU

Danh sách mã nguồn của PS-CPU được trình bày dưới đây.

subcpu								Thư mục PS-CPU
							PowerSubCPU.uvmpw	File project của PS-CPU
							common	Thư mục liên quan đến wrapper của driver
							Drivers	-
							BSP	-
							Inc	-
							BSP_STM32F030_Nucleo.h	File header cho board đánh giá dòng STM32F030
							Src	-
							BSP_GP10_STM32F030_Nucleo.c	File source cho board đánh giá dòng STM32F030
							CMSIS	-
							Device	-
							ST	-
							STM32F0xx	-
							Include	-
							Driver_Common.h	
							FLASH_stm32f0xx.h	File header của driver flash cho giao diện CMSIS
							I2C_STM32F030.h	File header của driver I2C cho giao diện CMSIS
							I2C_stm32f0xx.h	File header của driver I2C cho giao diện CMSIS
							Source	-
							FLASH_stm32f0xx.c	File source của driver flash cho giao diện CMSIS
							I2C_STM32F030.c	File source của driver I2C cho giao diện CMSIS
							I2C_stm32f0xx.c	File source của driver I2C cho giao diện CMSIS
							STM32F0xx_HAL_Driver	Driver HAL do CubeMX sinh ra, dùng chung cho iap và main (được di chuyển thủ công)
							Inc	
							stm32f0xx_hal.h	
							...	
							Src	
							stm32f0xx_hal.c	
							...	

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

iap									
	App								
		entry_iap.c							File do CubeMX sinh ra (ban đầu là main.c)
		km_i2c_iap.c							File source xử lý I2C do KM tạo
		km_extend_io_iap.c							Xử lý IO mở rộng của KM (phía IAPIAPMSW_ON)
	Inc								
		Header của hệ thống xử lý chính							Main processing system header
		km_i2c_iap.h							Header liên quan đến xử lý I2C của KM
		mxconstants.h							File do CubeMX sinh ra
		km_ver.h							File header thông tin phiên bản do KM tạo
		stm32f0xx_hal_conf.h							File do CubeMX sinh ra
		stm32f0xx_it.h							File do CubeMX sinh ra
	MDK-ARM								
		PowerSubCPU_iap.uvprojx							File project IAP
		PowerSubCPU_iap							-
		startup_stm32f030x8.s							File do CubeMX sinh ra
		PowerSubCPU_iap.ioc							CubeMX dùng cho IAP
	Src								
		mx_init.c							File thay thế cho main.c do CubeMX sinh ra
		stm32f0xx_hal_msp.c							File do CubeMX sinh ra
		stm32f0xx_it.c							File do CubeMX sinh ra
		system_stm32f0xx.c							File do CubeMX sinh ra
	main								
	App								
		entry.c							Main - xử lý chính
		km_extend_io.c							Xử lý I/O mở rộng do KM tạo
		km_i2c.c							Xử lý I2C do KM tạo
		km_it.c							Xử lý ngắt do KM tạo
		km_adc.c							Xử lý ADC do KM tạo
	Inc								
		entry.h							Main processing system header
		km_extend_io.h							Header liên quan đến I/O mở rộng
		km_i2c.h							Header liên quan đến xử lý I2C của KM
		km_adc.h							Header liên quan đến xử lý ADC của KM
		km_it.h							Header liên quan đến xử lý ngắt do KM tạo
		km_ver.h							File header thông tin phiên bản do KM tạo
		mxconstants.h							File do CubeMX sinh ra
		stm32f0xx_hal_conf.h							File do CubeMX sinh ra
		stm32f0xx_it.h							File do CubeMX sinh ra
	MDK-ARM								
		PowerSubCPU.uvprojx							Main - file project chính
		PowerSubCPU							
		startup_stm32f030x8.s							
		PowerSubCPU.ioc							CubeMX dùng cho main
	Src								
		mx_init.c							File do CubeMX sinh ra
		stm32f0xx_hal_msp.c							File do CubeMX sinh ra
		stm32f0xx_it.c							File do CubeMX sinh ra

4. 3. Môi trường phát triển

Môi trường phát triển của PS-CPU được mô tả dưới đây.

4. 3. 1. STM32CubeMX

STM32CubeMX là công cụ sinh mã nguồn do STMicro cung cấp.

Bằng GUI, có thể cấu hình gán chân và bật/tắt các ngoại vi; dựa trên các thiết lập đó, project cho Keil và các môi trường khác sẽ được sinh ra.

Sử dụng bộ cài đặt sau.

¥¥10.128.16.110¥data1¥29P.システム統合 SoC 開発¥03_BSP¥10.ツール¥00.SubCPU_tool¥STMmicro¥STM32Cube

SetupSTM32CubeMX-4.14.0.exe

Tiến hành cài đặt theo các màn hình hiển thị.

※Không có lưu ý đặc biệt nào.

7/25 Phiên bản đang sử dụng hiện tại là Version 4.15.1, vì vậy hãy khởi động STM32CubeMX một lần và kiểm tra như sau.

Help -> Check for update

Khi kiểm tra, Version 4.15.1 sẽ được hiển thị, hãy thực hiện Install.

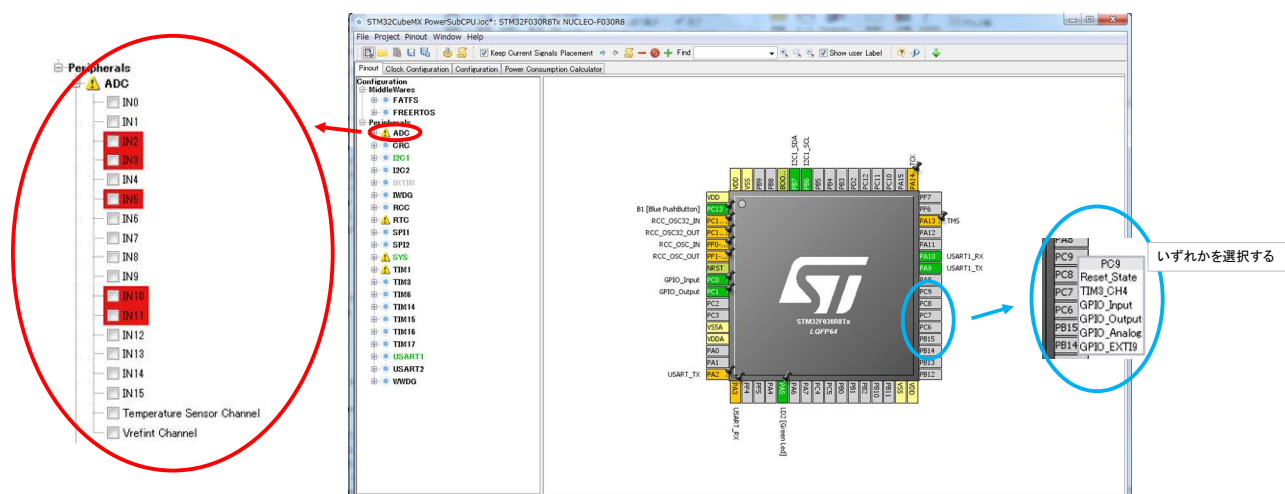
※Sau khi Install, hãy khởi động lại ứng dụng; tuy nhiên trong một số trường hợp có thể xảy ra lỗi khi khởi động lại. Nếu gặp trường hợp đó, hãy thử chạy ở chế độ administrator.

4. 3. 1. 1. Phương pháp cấu hình ngoại vi (Peripherals) và PIN

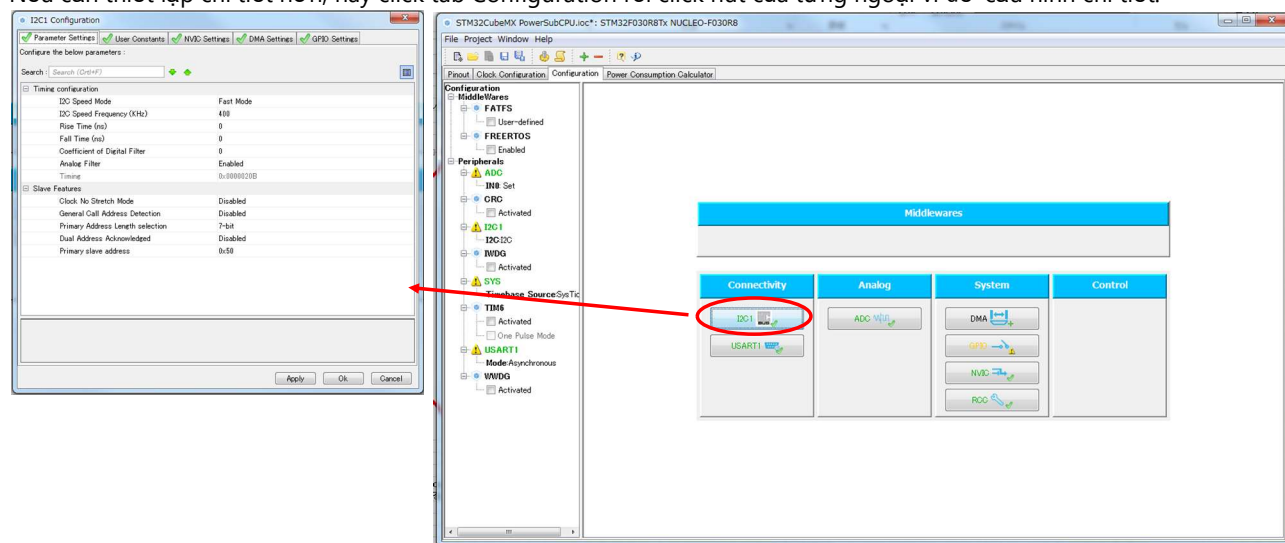
Khởi động STM32CubeMX rồi click tab Pinout để hiển thị màn hình sau.

Đề bật một ngoại vi, **như trong vùng khung đỏ**, hãy chọn mục đó và đánh dấu chọn để bật.

Đối với cấu hình PIN, **như trong vùng khung xanh**, hãy chọn PIN và chọn chức năng cần bật.



Nếu cần thiết lập chi tiết hơn, hãy click tab Configuration rồi click nút của từng ngoại vi để cấu hình chi tiết.



4.3.1.2. Sinh mã (Code generation)

Sau khi hoàn tất thiết lập cho từng ngoại vi, click Project->Generate Code để sinh mã.

Mã được sinh ra tại vị trí chứa file *.ioc (file project của STM32CubeMX).

Đối với PS-CPU của S800, project STM32CubeMX được cung cấp riêng cho main project và iap project, và phải sinh mã cho từng project.

Khi thực hiện sinh mã, main.c được tự động sinh ra, nhưng đối với PS-CPU của S800 thì file này không được dùng nguyên trạng; nó phải được gộp (merge) với mx_init.c.

Lý do main.c không được dùng nguyên trạng là vì hàm main() cần được tùy chỉnh, trong khi về cơ bản chúng ta không muốn sửa mã do STM32CubeMX sinh ra.

Hàm main() được định nghĩa trong entry.c do KM tạo.

4.3.2. µVision

µVision là môi trường phát triển tích hợp do ARM (Keil) cung cấp.

Hỗ trợ trình biên dịch, quản lý mã nguồn và gỡ lỗi sử dụng ulink.

Sử dụng bộ cài đặt sau.

[¥10.128.16.110¥data1¥29U.次世代システム検証環境開発¥01_共通¥10.ツール¥MDK-ARM](#)

MDK515.EXE

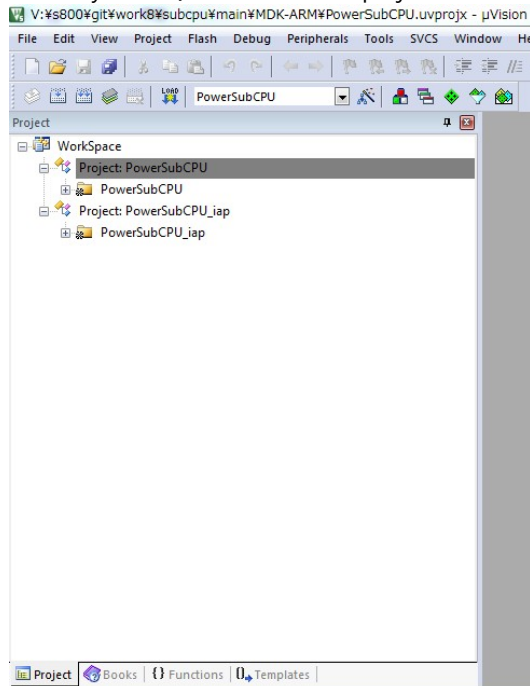
Tiến hành cài đặt theo các màn hình hiển thị.

※Không có lưu ý đặc biệt nào.

4.3.2.1. Multi-project (nhiều dự án)

Đối với PS-CPU của S800, µVision được dùng để quản lý hai project (main và iap) trong cùng một project.

Tại thư mục gốc dùng cho quản lý mã nguồn, PowerSubCPU.uvmpw khi khởi động, màn hình như sau sẽ xuất hiện và có thể thấy nó được chia thành hai project.



Để chuyển đổi giữa hai project, click chuột phải vào tên project (project có 🌟 đính kèm). "Set as Active Project" sẽ được hiển thị, hãy click vào đó.

Khi đó project này trở thành active và có thể thực hiện build, debug, v.v.

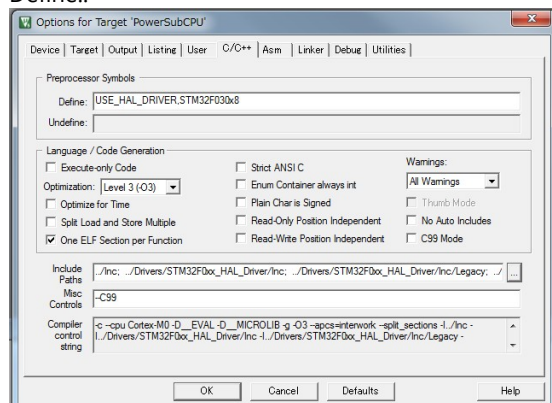
4.3.2.2. Build

Từ tab Project của µVision, có thể thực hiện build.

Khi dùng multi-project, hãy chú ý project nào đang được chọn.

Nếu muốn thêm tùy chọn build, click chuột phải vào thư mục nằm ngay dưới mỗi project (tên project có 🌟 đính kèm), chọn thư mục ngay bên dưới nó, rồi click chuột phải và chọn "Options for Target có tên – Target 'Tên Target'".

Màn hình sau sẽ xuất hiện; hãy chọn tab C/C++. Thêm các tùy chọn build mong muốn vào Preprocessor Symbols tại mục Define:..



4.3.2.3. Trình gỡ lỗi (Debugger)

Để kết nối tới CPU, thực hiện Debug->Start/Stop Debug Session.

Khi thực hiện, nó sẽ kết nối tới CPU và ghi các file binary do từng project sinh ra vào flash ROM.

※ Khi kết nối tới CPU, việc nạp vào flash ROM luôn được thực hiện.

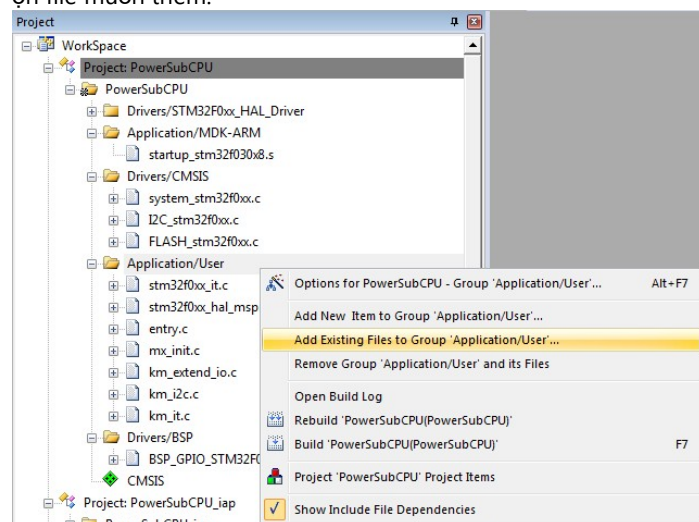
Theo mặc định, thay vì dừng tại reset vector, chương trình chạy tới đầu hàm main() và dừng (break) tại đó.

Cách gỡ lỗi giống với trình gỡ lỗi thông thường nên có thể sử dụng một cách trực quan.

Nếu chỉ muốn ghi lại flash ROM, thực hiện Flash->Download. CPU sẽ không được debug; chỉ dữ liệu trong ROM bị thay đổi.

4.3.2.4. Thêm file

Để thêm file, chọn thư mục muốn thêm vào rồi click chuột phải. Chọn "Add Existing File to Group <tên thư mục>" rồi chọn file muốn thêm.



Các file được thêm ở đây sẽ tự động được đưa vào quá trình build.

5. Đặc tả điều khiển

Việc điều khiển PS-CPU được thực hiện dựa trên đặc tả điều khiển power sub-CPU.

[Tài liệu tham khảo]

Tài liệu liên quan 2: Power Sub-CPU Control Specification (XX).docx

5. 1. Tổng quan điều khiển SW

5. 1. 1. Tín hiệu đầu vào của KM

Các tín hiệu MFP được dùng làm đầu vào cho PS-CPU như sau.

※ Về ngắt và đầu vào analog, xem chương riêng.

Loại chân	Chức năng	Tên tín hiệu
PA8	GPIO_Input	MONI_24V11
PA11	GPIO_Input	SB_PG
PA15	GPIO_Input	MC3.3VON_MONI

5. 1. 1. 1. MONI_24V11

Sử dụng bộ so sánh (comparator) để xác định nguồn DC24V11 (engine) có đạt hoặc vượt giá trị quy định hay không.

※ Trong đặc tả Sub-CPU nguồn, tên tín hiệu là 24V11_MONI nhưng khi phát triển phần mềm, do định nghĩa hằng số không thể bắt đầu bằng chữ số nên tên tín hiệu đã được thay đổi.

MONI_24V11 khi ở mức Low nghĩa là việc xả (discharge) 24V đã hoàn tất, khi ở mức High nghĩa là việc xả chưa hoàn tất.

Từ S800, MC_P_ON(IR_P_ON) Ngay cả khi có yêu cầu đặt lên High, nếu quá trình xả chưa hoàn tất thì phải chặn việc xuất MC_P_ON(IR_P_ON) và chờ xả hoàn tất, sau đó mới xuất MC_P_ON(IR_P_ON).

Khi khởi động, trước hết chờ xả hoàn tất, sau đó bật nguồn cho S800 sau 500 ms kể từ khi xả hoàn tất.

5. 1. 1. 2. SB_PG

Tín hiệu Power Good của khối AON trên S800.

Khi SB_PG ở mức Low, nguồn của khối AON bị ngắt.

Khi SB_PG ở mức High, đang phát hiện Power Good.

5. 1. 1. 3. MC3.3VON_MONI

Tín hiệu Power Good phát ra từ nguồn 3.3 V của khối mech-con.

Khi MC3.3VON_MONI chuyển sang Low, chờ 0.5 giây trôi qua rồi xác định rằng nguồn của khối mech-con đã bị ngắt.

Khi MC3.3VON_MONI ở mức High, đang phát hiện Power Good.

PS-CPU không thực hiện điều khiển nào được kích hoạt bởi tín hiệu này.

Nếu trong tương lai cần điều khiển khối mech-con thì tín hiệu này sẽ được sử dụng.

5.1.2. Tín hiệu đầu ra của KM

Các tín hiệu được dùng làm đầu ra từ PS-CPU như sau.

Loại chân	Chức năng	Tên tín hiệu
PB0	GPIO_Output	IR_P_ON
PB1	GPIO_Output	ERP_SENSOR_ON
PB2	GPIO_Output	/RESET
PB3	GPIO_Output	SB_PWR_EN
PB5	GPIO_Output	MC_P_ON
PB8	GPIO_Output	/DISCHG
PB9	GPIO_Output	/RST_SLP2
PB10	GPIO_Output	/USB2_OE
PB13	GPIO_Output	MC_PWR_EN
PC13	GPIO_Output	WAKEUP

5.1.2.1. IR_P_ON

Tín hiệu điều khiển relay nguồn IR.

Khi khởi động, đồng thời với /RESET, đặt IR_P_ON lên High.

Khi phát hiện POWER_MONITOR ở mức Low, lập tức đặt IR_P_ON xuống Low.

Ngoài hai trường hợp trên, việc điều khiển được thực hiện theo điện văn (telegram) từ LPPP (2nd Địa chỉ 0x70: PB0).

Tuy nhiên, khi đặt lên High, 24V11_MONI hãy chờ tín hiệu ở mức Low (trạng thái đã xả 24 V) rồi mới đặt lên High.

5.1.2.2. ERP_SENSOR_ON

Tín hiệu ngắt nguồn cho các cảm biến liên quan đến engine (DOOR_OPEN, DEST_EDH, CLOSE).

Ở trạng thái ErP lot.6 và ở trạng thái SLEEP2 có thiết lập tắt cảm biến, ERP_SENSOR_ON xuất mức Low, ngoài ra xuất mức High.

Vì PS-CPU không nhận biết trạng thái tiết kiệm điện nên nó phán đoán dựa trên tín hiệu AP_PWR_EN.

Khi AP_PWR_EN ở mức Low và điện văn (2nd Địa chỉ 0x70: PB1) ở mức Low, ERP_SENSOR_ON xuất mức Low.

Khi AP_PWR_EN ở mức High hoặc điện văn (2nd Địa chỉ 0x70: PB1) ở mức High, ERP_SENSOR_ON xuất mức High.

5.1.2.3. /RESET

Tín hiệu reset xuất ra S800 và các mạch ngoại vi.

Có ba thời điểm điều khiển như sau.

Ngay sau khi PS-CPU khởi động, xuất /RESET ở mức Low

Sau khi MSW_ON lên High, xuất SB_PWR_EN High, rồi xuất /RESET High để giải phóng reset.

Khi phát hiện /HRESET_REQ ở mức Low, lập tức xuất /RESET Low.

5.1.2.4. SB_PWR_EN

Tín hiệu điều khiển nguồn cho khối AON của S800.

Khi phát hiện MSW_ON ở mức High, xuất SB_PWR_EN High.

Khi phát hiện /HRESET_REQ ở mức Low, xuất SB_PWR_EN Low.

Tuy nhiên, là hành vi chung cho cả hai điều kiện kích hoạt trên, nếu MONI_24V11 đang ở mức High thì chờ MONI_24V11 xuống Low (hoàn tất xả 24V), MONI_24V11 Low rồi 0.5 giây sau mới xuất SB_PWR_EN High.

5.1.2.5. MC_P_ON

Tín hiệu điều khiển relay nguồn cho MC (khối MechaCon).

Khi khởi động, đồng thời với /RESET, đặt MC_P_ON lên High.

Khi phát hiện POWER_MONITOR ở mức Low, lập tức đặt MC_P_ON xuống Low.

Ngoài hai trường hợp trên, điều khiển theo điện văn (2nd Địa chỉ 0x70: PB5).

Tuy nhiên, khi đặt lên High, 24V11_MONI chờ mức Low (trạng thái đã xả 24V) rồi mới đặt lên High.

5.1.2.6. /DISCHG

Không có kế hoạch sử dụng

5.1.2.7. /RST_SLP2

Tín hiệu reset cho các linh kiện ngoại vi của AP.

Vì các ngoại vi của AP không có reset từ bên ngoài nên PS-CPU thực hiện việc reset các linh kiện ngoại vi của AP.

Điện văn (2nd Địa chỉ 0x70: PB5) ở mức Low hoặc khi phát hiện AP_PWR_EN Low, xuất /RST_SLP2 ở mức Low.

Điện văn (2nd Địa chỉ 0x70: PB5) ở mức High và AP_PWR_EN High thì xuất /RST_USB High.

5.1.2.8. /USB2_OE

Đặc tả chưa xác định (T.B.D *Sẽ được mô tả sau khi đặc tả xử lý proxy cho wireless LAN được quyết định(9/Edur kiến))

5.1.2.9. MC_PWR_EN

Tín hiệu điều khiển nguồn cho các IC xung quanh MC (khối MechaCon).

Khi phát hiện MONI_24V11 ở mức Low, xuất MC_PWR_EN Low.

MC_PWR_EN High được xuất ra đồng thời với MC_P_ON High.

5.1.2.10. WAKEUP

Tín hiệu khởi động do PS-CPU phát ra, gửi tới AP của S800.

Khi nhận được ngắt từ RTC tích hợp trong PS-CPU, WAKEUP High được xuất ra.

Vì tín hiệu này được nối trực tiếp tới ngắt RTC Alarm tích hợp trong PS-CPU nên phần mềm PS-CPU không cần thiết. Phía S800 sẽ trực tiếp điều khiển RTC.

*Các ngắt Wakeup khác trong S72/73 do LPPP điều khiển.

5.1.3. Tín hiệu ngắt của KM

Các tín hiệu ngắt đi vào PS-CPU từ bên ngoài như sau.

Số chân	Loại chân	Chức năng	Tên tín hiệu	Thiết lập ngắt
11	PA1	GPIO_EXTI1	/HRESET_REQ	External Interrupt Mode with Falling edge trigger detection
12	PA2	GPIO_EXTI2	AP_PWR_EN	External Interrupt Mode with Rising/Falling edge
14	PA4	GPIO_EXTI4	POWER_MONITOR	External Interrupt Mode with Falling edge trigger detection
15	PA5	GPIO_EXTI5	MSW_ON	External Interrupt Mode with Rising/Falling edge

5.1.3.1. /HRESET_REQ

Tín hiệu yêu cầu khởi động lại do S800 phát ra.

Khi MSW_ON ở mức High, tại thời điểm /HRESET chuyển High → Low, chuỗi khởi động lại (restart sequence) được bắt đầu.

Khi MSW_ON ở mức Low, khi /HRESET chuyển High → Low, xử lý ngắt nguồn được bắt đầu.

Khi /HRESET chuyển Low → High, được xem là S800 đã khởi động xong (PS-CPU không làm gì).

5.1.3.2. AP_PWR_EN

Tín hiệu do LPPP phát ra để điều khiển nguồn của AP.

Trên PS-CPU, tín hiệu này được dùng cho thời điểm Low/High của tín hiệu ERP_SENSOR_ON.

Chi tiết xem [5.2.2.2. ERP_SENSOR_ON](#) để tham khảo

5.1.3.3. POWER_MONITOR

Tín hiệu phát hiện DC5V do bộ nguồn điện áp thấp phát ra

Khi POWER_MONITOR chuyển Low → High, không làm gì cả.

Khi POWER_MONITOR chuyển High → Low, xuất vô điều kiện MC_P_ON Low và IR_P_ON Low.

5.1.3.4. MSW_ON

Tín hiệu biểu thị trạng thái của công tắc nguồn chính.

Nguồn LIVE cấp điện cho PS-CPU sẽ không bị ngắt chừng nào điện lưới còn được cấp, ngay cả khi công tắc nguồn chính được tắt.

Trong khi công tắc nguồn chính đang tắt, PS-CPU ở chế độ "Sleepmode" để giảm tiêu thụ điện năng.

[1] Khi phát hiện ngắt với MSW_ON=1

1. Hủy chế độ "Sleepmode" của chính nó.

2. Bật nguồn SB cho S800. (SB_PWR_EN=1)

3. Sau một khoảng thời gian quy định, giải phóng reset của SB. (RESETn=1)

4. Khởi động relay engine và nguồn 3.3 V cho MC.

(MC_P_ON=1, IR_P_ON=1, MC_PWR_EN=1)

[2] Khi phát hiện ngắt với MSW_ON=0 và SB_PWR_EN=0 (S800 đã dừng / đang khởi động lại)

1. Hủy bỏ tiến trình khởi động lại và ngắt toàn bộ nguồn trừ nguồn LIVE.
2. Sau khi xác nhận PG của tất cả các nguồn trừ LIVE đã xuống, chuyển chính nó sang "Sleepmode".

[3] Khi phát hiện ngắt với MSW_ON=0 và SB_PWR_EN=1

1. Ngắt relay engine. (MC_P_ON=0 và IR_P_ON=0)
2. Chờ HRESET_REQn=0 từ S800.
3. Khi phát hiện HRESET_REQn=0, ngắt toàn bộ nguồn trừ nguồn LIVE.
4. Sau khi xác nhận PG của tất cả các nguồn trừ LIVE đã xuống, chuyển chính nó sang "Sleepmode".

5. 1. 4. Điều khiển I2C

Việc giao tiếp với CA72 hoặc LPPP được thực hiện bằng I2C.

Do đặc tính của truyền thông I2C, CA72 và LPPP là master, còn phía PS-CPU được coi là thiết bị slave.

5. 1. 4. 1. Đặc tả truyền thông

Đặc tả truyền thông của PS-CPU như sau.

Chế độ bus I2C	Fast Mode (400KHz)
Độ rộng địa chỉ	7 bit(không tính bit R/W))
Địa chỉ slave	0x3D (RTC)
	0x3E(không tính RTC))

Trong truyền thông I2C, phía master (CA72, LPPP) luôn gửi địa chỉ slave của thiết bị (PS-CPU) rồi đến địa chỉ thứ 2nd (2nd address), sau đó thực hiện ghi hoặc đọc dữ liệu.

Trong truyền thông I2C, sau địa chỉ thứ 2nd (2nd address) 8 bit, các đặc tả sau được áp dụng.

- Khi ghi vào PS-CPU, ghi 4 byte dữ liệu.
- Khi đọc từ PS-CPU, đọc 2 byte dữ liệu.

*Tuy nhiên, dữ liệu chương trình cho yêu cầu ghi lại chương trình là 1024 byte.

Lý do của đặc tả này là trong các driver thông thường, truyền theo khối (block transfer) được hỗ trợ cho Write, nhưng trong nhiều trường hợp lại không được hỗ trợ cho Read.

Chi tiết về READ và WRITE, tham khảo Đặc tả điều khiển Power Sub-CPU.

※10.3.1. Ghi từ CPU chính

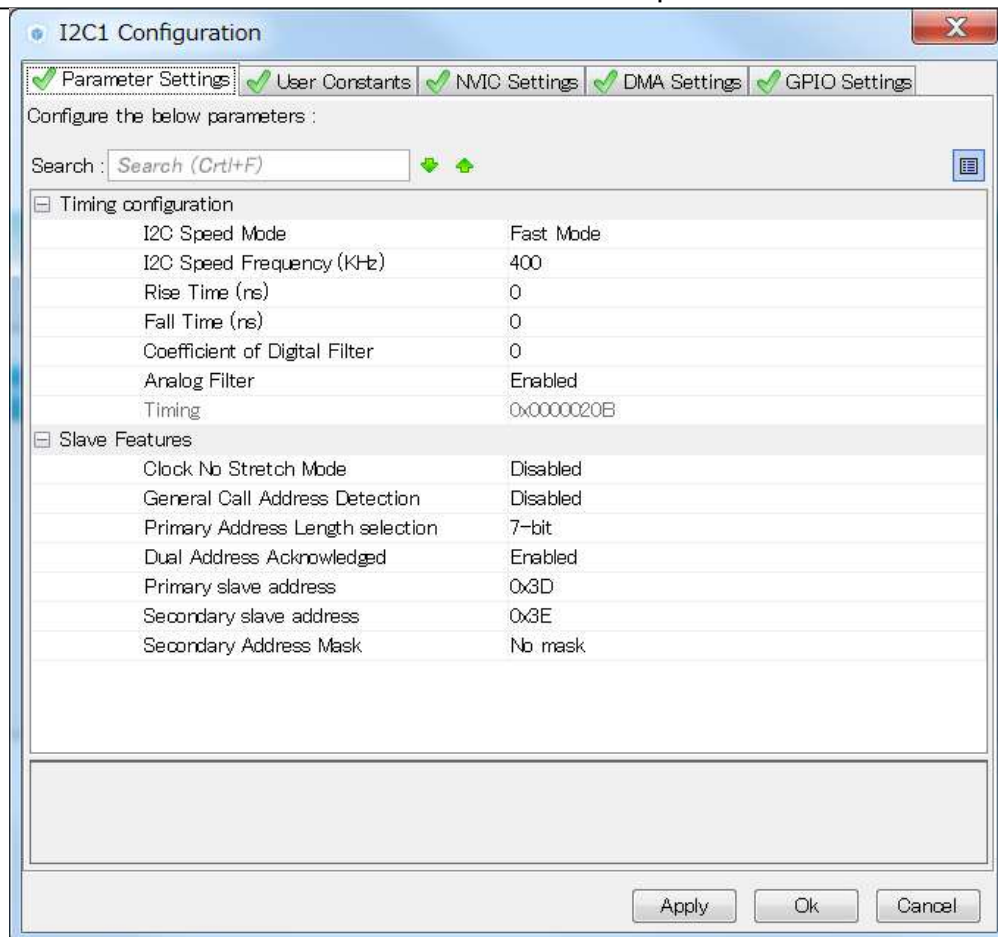
※10.3.2. Đọc bởi CPU chính

[Tài liệu tham khảo]

Tài liệu liên quan 2: Power Sub-CPU Control Specification (XX).docx

5. 1. 4. 2. Thiết lập STM32CubeMX

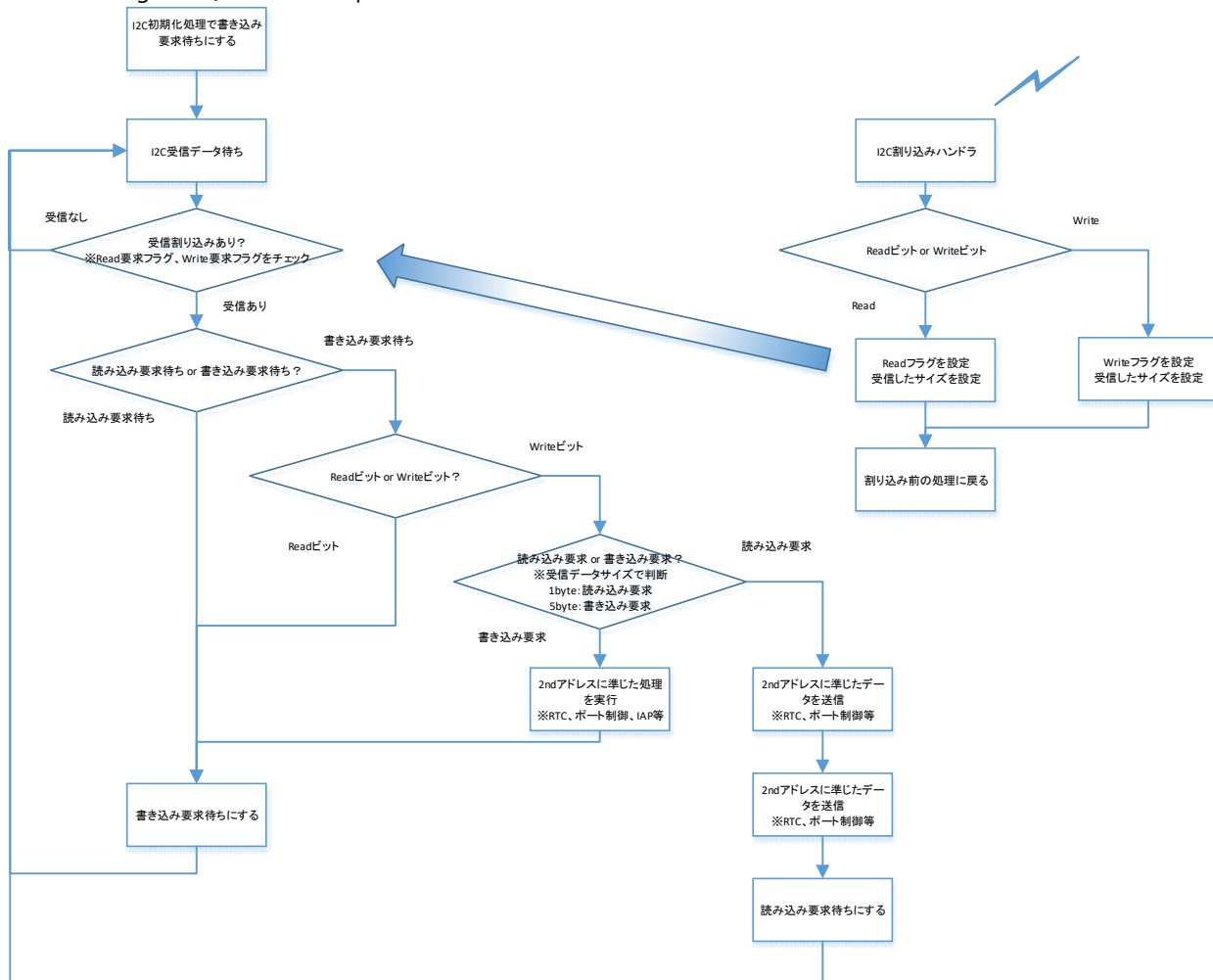
Thiết lập STM32CubeMX như hình dưới đây.



5.1.4.3. Trình tự điều khiển

Trình tự điều khiển của PS-CPU được trình bày dưới đây.

Việc nhận dữ liệu được thực hiện bằng ngắt; cho tới khi ngắt xảy ra thì chỉ kiểm tra cờ ngắt, nhờ vậy đặc tả này giảm thiểu ảnh hưởng tới việc điều khiển port.



Như đã mô tả trong đặc tả truyền thông I2C của Đặc tả điều khiển Power Sub-CPU, master (S800) luôn ghi địa chỉ 2nd, và yêu cầu Write chính là điểm khởi đầu.

Khi ghi vào đĩa chỉ 2nd, việc truyền thông được thực hiện với đĩa chỉ 2nd (1 byte) + dữ liệu ghi (4 byte) + Stop bit.

Khi đọc từ đĩa chỉ 2nd, sau đĩa chỉ 2nd (1 byte) + Stop bit, phía slave (PS-CPU) gửi dữ liệu đọc + Stop bit.

[Tài liệu tham khảo]

Tài liệu liên quan 2: Power Sub-CPU Control Specification (XX).docx

5.1.4.4. PS-CPU 2nd address – Đặc tả**5.1.5.3.1. RTC**

Danh sách các thanh ghi điều khiển RTC.

*Giống với cấu hình thanh ghi RTC của STM32F031

2nd	0x00	0x04	0x08	0x0C	0x10	0x14	0x1C
Công dụng	RTC Time (thời gian)	RTC Date (ngày tháng)	RTC Control (điều khiển)	RTC – khởi tạo và trạng thái	RTC Pre-scaler (bộ chia tần)	RTC Wakeup Timer (bộ định thời đánh thức)	RTC Alarm A
0	SU[3:0]	DU[3:0]	WUCKSEL[2:0]	ALRAWF	PREDIV_S[14:0]	WUT[15:0]	SU[3:0]
1				ALRBWF			
2				WUTWF			
3				TSEDGE			
4	ST[2:0]	DT[1:0]	REFCKON	INTS			ST[2:0]
5			BYPSHAD	RSF			
6			FMT	INTF			
7				INT			
8	MNU[3:0]	MU[3:0]	ALRAE	ALRAF			MNU[3:0]
9				WUPF			
10			WUTE				
11			TSE	TSF			
12	MNT[2:0]	MT	ALRAIE	TSOVF			MNT[2:0]
13		WDU[2:0]		TAMP1F			
14			WUTIE	TAMP2F			
15			TSIE	TAMP3F			
16	HU[3:0]	YU[3:0]	ADD1H	RECALPF	PREDIV_A[6:0]		HU[3:0]
17			SUB1H				
18			BKP				
19			COSEL				
20	HT[1:0]	YT[3:0]	POL				HT[1:0]
21			OSEL[1:0]				
22			COE				
23							
24							DU[3:0]
25							
26							
27							
28							DT[1:0]
29							
30							WDSEL
31							MASL4

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

2nd	0x24	0x28	0x2C	0x30	0x34	0x38	0x3C
Công dụng	RTC – bảo vệ ghi (Write Protection)	RTC Sub Second (phần giây lẻ)	RTC Shift Control	RTC Timestamp Time	RTC Timestamp Date	RTC Timestamp Sub-second	RTC – hiệu chỉnh (Calibration)
0	KEY[7:0]	SS[15:0]	SUBFS[14:0]	SU[3:0]	DU[3:0]	SS[15:0]	CALM[8:0]
1							
2							
3							
4				ST[2:0]	DT[1:0]		
5							
6							
7							
8	MNU[3:0]	MU[3:0]					
9							
10							
11							
12	MNT[2:0]	MT					
13		WDU[2:0]					
14							
15							
16			HU[3:0]				
17							
18							
19							
20			HT[1:0]				
21							
22			PM				
23							
24							
25							
26							
27							
28							
29							
30							
31							

5.1.5.3.2. Trạng thái nội bộ (Internal Status)

2nd	0x68	0x6C
Công dụng	Trạng thái nội bộ	Dự trữ (Reserved)
0	Trạng thái khởi động 0: Khởi động nguội (Cold boot) 1: Khởi động lại (Reboot)	
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		
20		
21		
22		
23		
24		
25		
26		
27		
28		
29		
30		
31		

5.1.5.3.3 Mở rộng I/O

Các tín hiệu cần thiết cho I/O trong MFP.

2nd	0x70	0x74	0x78	0x7c
Công dụng	Mở rộng đầu ra Thanh ghi đọc ※Chỉ đọc (Read Only)	Thanh ghi đặt bit mở rộng đầu ra lên High ※Chỉ ghi (Write Only)	Thanh ghi đặt bit mở rộng đầu ra xuống Low ※Chỉ ghi (Write Only)	Dự trữ (Reserved)
0	PB0 IR_P_ON	PB0 IR_P_ON	PB0 IR_P_ON	
1	PB1 /ERP(sensor on)	PB1 /ERP(sensor on)	PB1 /ERP(sensor on)	
2	PB2 /RESET	PB2 /RESET	PB2 /RESET	
3	PB3 -	PB3 -	PB3 -	
4	PB4 -	PB4 -	PB4 -	
5	PB5 MC_P_ON	PB5 MC_P_ON	PB5 MC_P_ON	
6	PB6 -	PB6 -	PB6 -	
7	PB7 -	PB7 -	PB7 -	
8	PB8 -	PB8 -	PB8 -	
9	PB9 /RST_SLP2	PB9 /RST_SLP2	PB9 /RST_SLP2	
10	PB10 -	PB10 /ENG_STATUS1	PB10 /ENG_STATUS1	
11	PB11 -	PB11 /ENG_STATUS2	PB11 /ENG_STATUS2	
12	PB12 -	PB12 -	PB12 -	
13	PB13 -	PB13 -	PB13 -	
14	PB14 -	PB14 -	PB14 -	
15	PB15 -	PB15 -	PB15 -	
16	PB16	PB16	PB16	
17	PB17	PB17	PB17	
18	PB18	PB18	PB18	
19	PB19	PB19	PB19	
20	PB20	PB20	PB20	
21	PB21	PB21	PB21	
22	PB22	PB22	PB22	
23	PB23	PB23	PB23	
24	PB24	PB24	PB24	
25	PB25	PB25	PB25	
26	PB26	PB26	PB26	
27	PB27	PB27	PB27	
28	PB28	PB28	PB28	
29	PB29	PB29	PB29	
30	PB30	PB30	PB30	
31	PB31	PB31	PB31	

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

2nd	0x80	0x84	0x88	0x8C
Công dụng	Kiểm tra bit đầu vào	Dự trữ (Reserved)	Main checksum (2Byte)	Kiểm tra phiên bản
0	PA0 -		Giá trị checksum (2Byte)	Minor[3:0]
1	PA1 /HRESET_REQ			
2	PA2 AP_PWR_EN			
3	PA3 -			
4	PA4 POWER_MONITOR			Minor[7:4]
5	PA5 -			
6	PA6 -			
7	PA7 MC_OE_REQ			
8	PA8 24V11_MONI			Major[11:8]
9	PA9 -			
10	PA10 MSW_ON			
11	PA11 -			
12	PA12 -			Major[15:12]
13	PA13 -			
14	PA14 -			
15	PA15 MC3.3VON_MONI			
16				
17				
18				
19				
20				
21				
22				
23				
24				
25				
26				
27				
28				
29				
30				
31				

5.1.5.3.4 Ghi lại chương trình

Thanh ghi dùng cho việc ghi lại chương trình.

2nd	0x90	0x94	0x98	0x9C	0xA0
Công dụng	Yêu cầu ghi lại chương trình	Hoàn tất ghi lại dữ liệu chương trình	Trạng thái ghi lại chương trình	Dự trữ (Reserved)	Dữ liệu ghi lại chương trình
0	Magic number: 0x11223344 Hãy đặt giá trị này. Nếu nhận được dữ liệu khác với magic number nói trên, yêu cầu ghi chương trình sẽ không được chấp nhận		Trạng thái chuyển sang IAP 0: Chưa hoàn tất 1: Đã hoàn tất		Dữ liệu ghi lại chương trình được nhận dưới dạng dữ liệu nhị phân theo đơn vị 1024 byte. Dữ liệu được nhận theo thứ tự bắt đầu từ địa chỉ đầu tiên của Flash ROM.
1			Trạng thái dữ liệu ghi lại chương trình 0: Chưa hoàn tất 1: Đã hoàn tất		
2		Hoàn tất dữ liệu chương trình 0: Chưa hoàn tất 1: Đã hoàn tất	Trạng thái hoàn tất dữ liệu chương trình 0: Chưa hoàn tất 1: Đã hoàn tất		
3					
4					
5					
6					
7					
8		Checksum (1 byte) Giá trị thu được bằng cách cộng dồn từng byte của dữ liệu chương trình			
9					
10					
11					
12					
13					
14					
15					
16			Bit lỗi checksum 0: Không có lỗi xảy ra 1: Có lỗi xảy ra		
17			Lỗi nội bộ của PS-CPU 0: Không có lỗi xảy ra 1: Có lỗi xảy ra		
18					
19					
20					
21					
22					
23					
24					
25					
26					
27					
28					
29					
30					
31					

5.1.5. Điều khiển IAP

Điều khiển IAP là việc điều khiển ghi lại chương trình của chính PS-CPU trong khi PS-CPU đang chạy.

5.1.5.1. Cơ sở của điều khiển IAP

Cơ sở của điều khiển IAP là chương trình mẫu IAP do Hakuto cung cấp.

Chương trình mẫu như sau.

< ¥¥10.128.16.110¥data1¥29P.System Integration SoC Development¥03_BSP¥07.Outsourcing Management¥Hakuto (PS-CPU)¥From Hakuto¥20160614_Question List, IAP Sample (STM32F030R8) >

IAP_Sample

Chương trình mẫu được thiết kế để nhận dữ liệu qua truyền thông UART và thay đổi chương trình.

Phần thực sự được tham khảo là ý tưởng phân tách project và quá trình chuyển từ main project sang iap project.

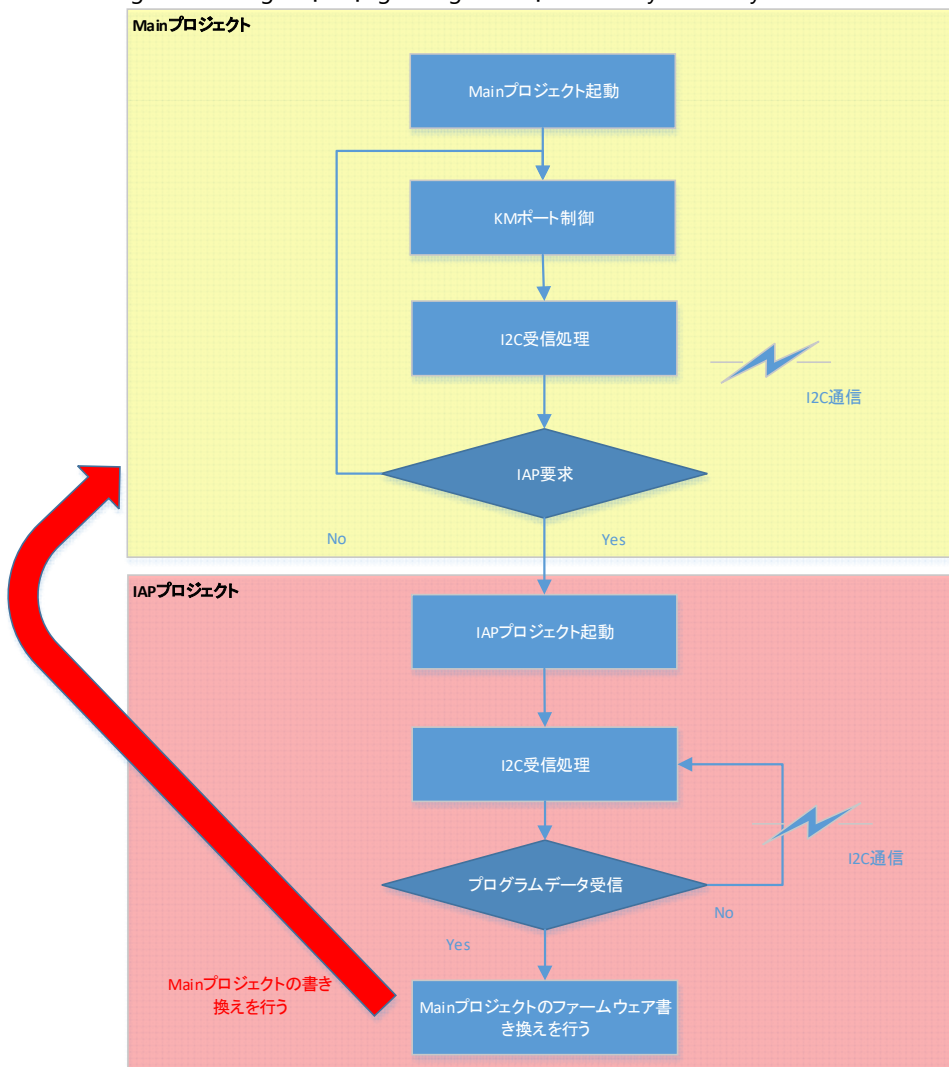
5.1.5.2. Phân tách project

Đối với PS-CPU của S800, điều khiển IAP được triển khai bằng cách tách thành hai project.

Tổng quan về các project như sau.

- ① main project hoạt động theo đặc tả điều khiển power Sub-CPU.
- ② iap project chỉ thực hiện các thao tác cần thiết cho việc ghi lại main project.

Sơ đồ khối đơn giản và luồng hoạt động đơn giản được trình bày dưới đây.



main project khởi động, và khi nhận được yêu cầu IAP từ CA72 hoặc từ LPPP thì nhảy sang chương trình iap.

Chương trình iap nhận dữ liệu chương trình của main project và cập nhật main project.

Project iap chỉ có thể được ghi lại qua ICE, không thể ghi lại từ Subset.

5.1.5.3. Phân vùng FlashROM

5.1.5.1. Phân tách project Như đã mô tả, do tách thành main project và iap project nên cần phải chia vùng FlashROM nơi ghi dữ liệu chương trình.

Bản đồ bộ nhớ khái quát của STM32F031 như sau.

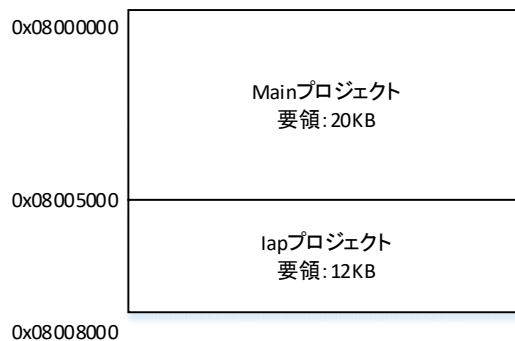
デバイス	境界アドレス	サイズ	メモリ領域	レジスタの説明
STM32F03x	0x2000 1000 - 0x3FFF FFFF	~512 MB	予約済み	
	0x2000 0000 - 0x2000 0FFF	4 KB	SRAM	セクション 2.3 (51 ページ)
	0x1FFF FC00 - 0x1FFF FFFF	1 KB	予約済み	
	0x1FFF F800 - 0x1FFF FBFF	1 KB	オプションバイト	セクション 4 (75 ページ)
	0x1FFF EC00 - 0x1FFF F7FF	3 KB	システムメモリ	
	0x0800 8000 - 0x1FFF EBFF	~384 MB	予約済み	
	0x0800 0000 - 0x0800 7FFF	32 KB	メインフラッシュメモリ	セクション 3 (55 ページ)
	0x0000 8000 - 0x07FF FFFF	~128 MB	予約済み	
	0x0000 0000 - 0x0000 7FFF	32 KB	メインフラッシュメモリ、システムメモリまたはSRAM (BOOT 設定に応じて)	

Khung đỏ là vùng lưu chương trình.

Vì có vùng 32KB nên main project dùng 20KB và iap project dùng 12KB.

※Tuy nhiên, nếu khối lượng chức năng của từng project tăng lên và kích thước chương trình lớn hơn tại một thời điểm nào đó, vùng nhớ sẽ được điều chỉnh khi cần.

Việc phân bổ như sau.



5.1.5.4. Truyền thông I2C

Khi nhận được điện văn yêu cầu ghi lại chương trình (2nd address: 0x90 và data: 0x11223344), nó chuyển sang chế độ IAP.

Sau khi chuyển sang chế độ IAP, xử lý port thông thường không được thực hiện.

Cho tới khi chuyển sang IAP có xảy ra thao tác chuyển chương trình nên cần chờ khoảng 5 ms.

Sau đó điện văn yêu cầu ghi lại chương trình (2nd address: 0xA0, data 1024 byte) được nhận, và 1024 byte này được dùng để cập nhật vùng chương trình của main project.

Dữ liệu gửi đi là dữ liệu nhị phân, với giả định rằng dữ liệu được gửi tuần tự bắt đầu từ địa chỉ đầu tiên của flash ROM.

Khi việc nhận dữ liệu chương trình từ CPU chính hoàn tất, CPU chính đặt cờ hoàn tất ghi lại chương trình (2nd address: 0x94, bit0=1) và đồng thời đặt giá trị checksum của dữ liệu chương trình.

Chờ việc ghi lại chương trình kết thúc, sau đó PS-CPU cập nhật trạng thái ghi lại chương trình (2nd address: 0x98, bit0=1).

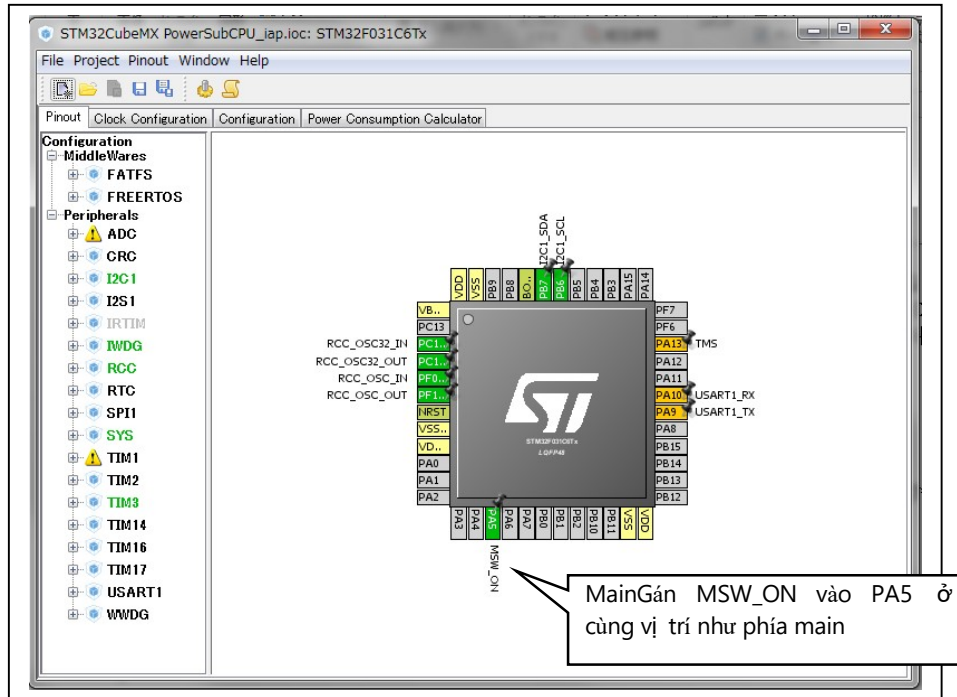
5.1.5.5. Reset khi ghi lại chương trình Điều khiển

Khi chạy iap project và ghi lại chương trình phía main, không thể chạy project phía main vừa được ghi lại nếu không thực hiện reset. Tuy nhiên, S800 theo đặc tả, ngay cả khi nguồn chính (MSW_ON) chuyển từ ON sang OFF thì PS-CPU vẫn tiếp tục chạy, và không thể reset trừ khi rút phích cắm điện. Do đó cần các biện pháp sau.

- Khi việc ghi lại chương trình hoàn tất, tự thực hiện software reset
- Để điều khiển thời điểm reset, cho phép phía IAP có thể phát hiện được MSW_ON

5.1.5.5.1 Hỗ trợ MSW_ON ở phía IAP

Khởi động STM32CubeMX và sửa thiết lập PIN ở phía IAP để GPIO MSW_ON có thể được phát hiện.



GPIO Pin Configuration được thiết lập như sau.

Pin Name : PA5
 Signal on Pn : n/a
 GPIO output level : n/a
 GPIO mode : External Interrupt Mode with Rising/Falling edge trigger detection
 GPIO Pull-up/Pull-down : No pull-up and no pull-down
 Maximum out speed : n/a
 Fast Mode : n/a
 User Label : MSW_ON
 Modified : Check

Việc sinh mã bằng thiết lập PIN giống như phía main, vì vậy hãy tham khảo [4.3.1.2. Sinh mã \(Code generation\)](#) Xem .

5.1.5.5.2 Hỗ trợ software reset

Khi phát hiện chuyển từ nguồn chính ON (MSW_ON=1) sang nguồn chính OFF (MSW_ON=0) trong lúc iap project đang chạy, thiết bị sẽ thực hiện software reset.

Để thực hiện software reset, đặt bit "SYSRESETREQ" trong thanh ghi Application Interrupt and Reset Control Register của Cortex-M0. Việc này được thực hiện bằng cách set bit đó.

"Trình tự thực hiện reset"

1. IAP Trong khi chương trình phía IAP đang chạy, nguồn chính ON (MSW_ON=1) ⇒ nguồn chính OFF (MSW_ON=0) được phát hiện
2. Thanh ghi hoàn tất ghi lại chương trình (0x94 2ndbit) đã được set
3. Khi cả điều kiện 1. và 2. đều thỏa mãn, gọi hàm NVIC_SystemReset() để thực hiện software reset
4. Khi software reset hoàn tất, chương trình nhảy tới hàm main() của main project.

※ Ngay cả khi phát hiện nguồn chính ON/OFF trong lúc việc ghi lại chưa hoàn tất, cũng không thực hiện gì đặc biệt.

5.1.6. Điều khiển RTC

RTC được điều khiển bằng cách CPU chính trực tiếp cấu hình các thanh ghi RTC của PS-CPU.

Do đó, các thanh ghi RTC được cho phép truy cập qua dải địa chỉ 2nd của I2C từ 0x00 đến 0x5C.

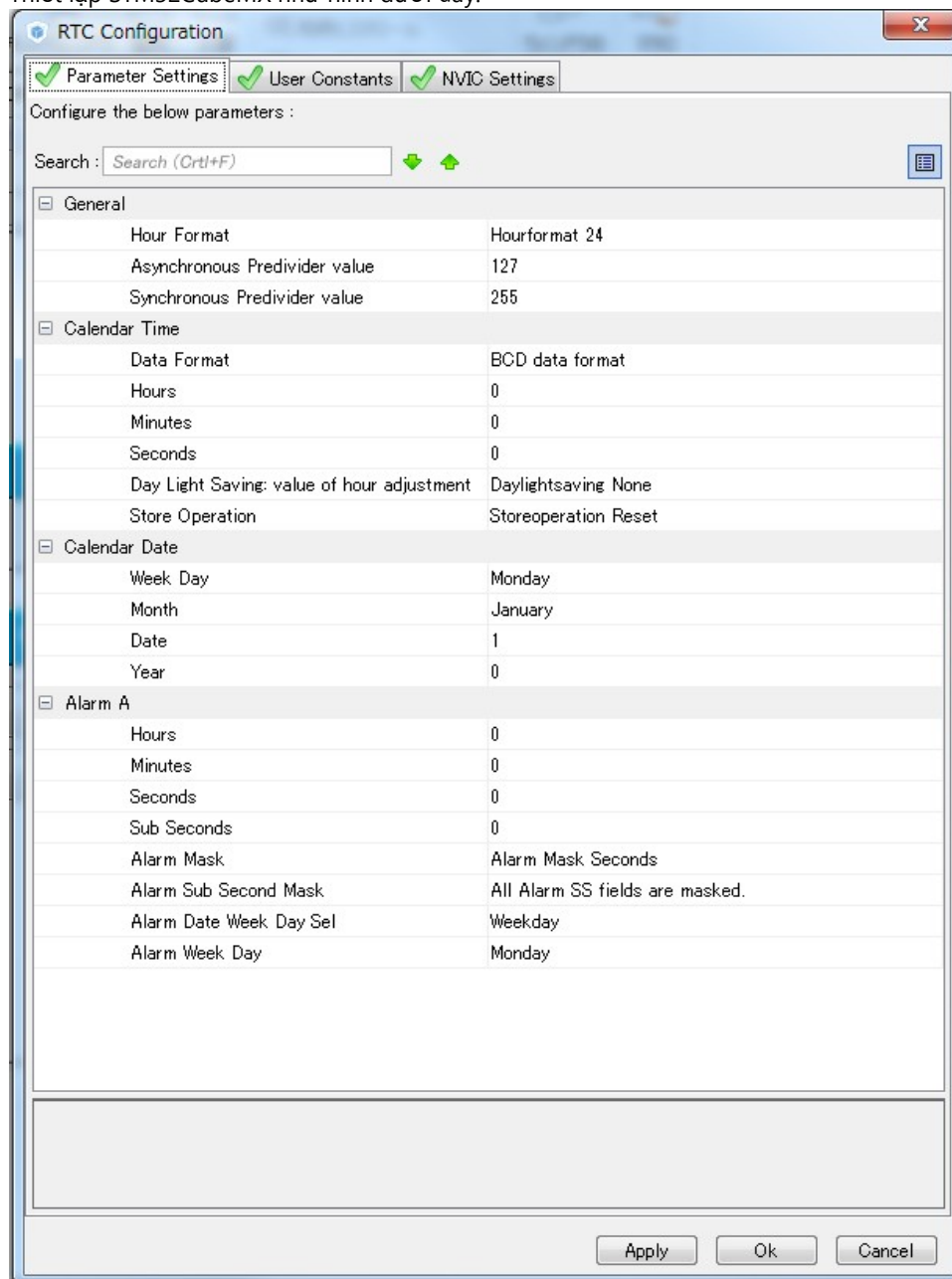
Chi tiết về thanh ghi, tham khảo datasheet của STM32F031.

[Tài liệu tham khảo]

Tài liệu tham khảo 1: STM32F0x1_2_8RM(RM0091)_Japanese_rev1.pdf

5.1.6.1. Thiết lập STM32CubeMX

Thiết lập STM32CubeMX như hình dưới đây.



Nếu tất cả thiết lập Alarm đều được cấu hình ở dạng mask, ngắt Alarm sẽ xảy ra mỗi chu kỳ clock — đây không phải hành vi mong muốn, nên thiết lập Alarm được sinh tự động trong mx_init.c được sửa lại như sau.

```
sAlarm.AlarmDateWeekDay = 0;
```

Bằng cách đặt Weekday bằng 0, nó trở thành thiết lập ngày trong tuần không tồn tại, và ngắt sẽ không còn xảy ra.

Đặt cấu hình trên làm cấu hình khởi tạo, còn thiết lập Alarm thực tế thì để phía CPU chính thực hiện.

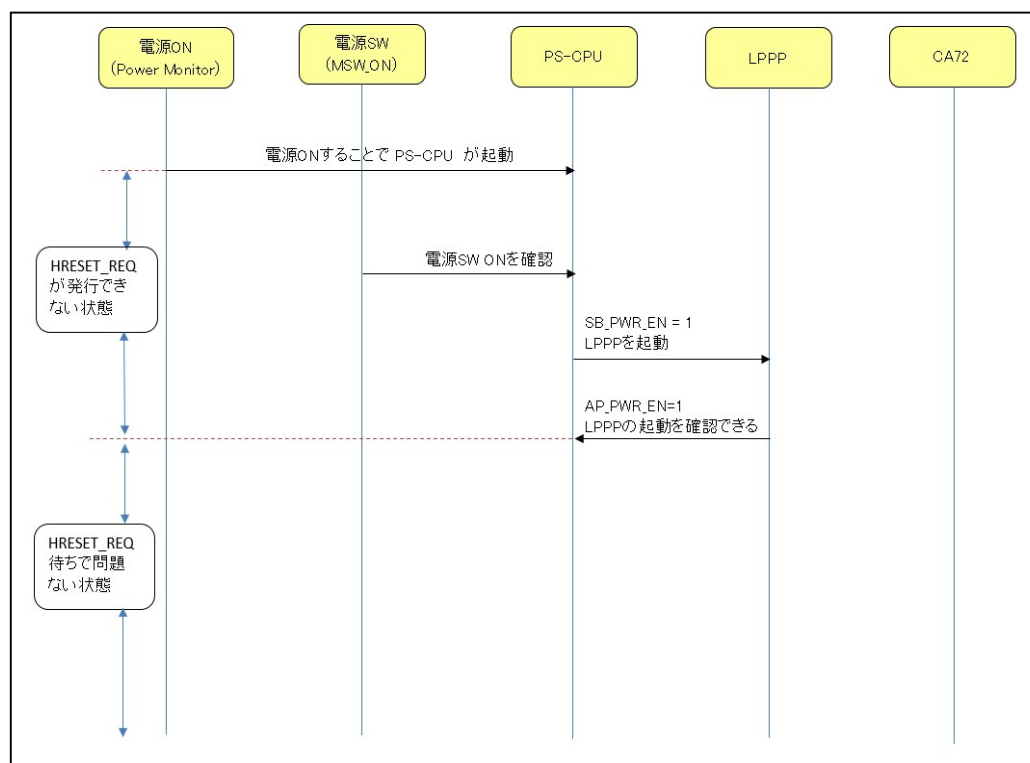
5.1.7. Xử lý mất điện tức thời khi bật nguồn lần đầu

Khi xảy ra mất điện tức thời thông thường, trình tự là: POWER_MONITOR xuống $\Rightarrow$ chờ HRESET_REQ $\Rightarrow$ trình tự tắt nguồn $\Rightarrow$ phục hồi khi POWER_MONITOR trở lại; ở phía PS-CPU, việc xử lý mất điện tức thời được thực hiện bằng cách dùng HRESET_REQ do phía AP phát ra làm điều kiện kích hoạt.

Tuy nhiên, khi vừa cắm phích cắm điện, tại lần bật nguồn đầu tiên, có trường hợp nguồn phía AP chưa ON. Khi đó chỉ có PS-CPU thức. Nếu mất điện tức thời xảy ra trong trạng thái này thì HRESET_REQ không thể được phát ra, nên chính PS-CPU phải thực hiện xử lý tắt nguồn.

5.1.7.1. Trình tự bật nguồn lần đầu

Dưới đây là trình tự đơn giản cho việc bật nguồn.



Như trong sơ đồ tuần tự ở trên, HRESET_REQ không thể được phát ra cho tới khi LPPP đặt AP_PWR_EN=1 và có thể xác nhận rằng LPPP đã khởi động. Trong khoảng thời gian đó, (nếu mất điện tức thời xảy ra khi AP_PWR_EN=0) thì chính PS-CPU phải thực hiện trình tự tắt nguồn.

Một khi AP_PWR_EN=1 và LPPP đã hoạt động, thì sau đó việc tuân theo HRESET_REQ không có vấn đề gì.

Ngoài ra, khi mất điện tức thời xảy ra trong DeepSleep/ERP, AP_PWR_EN trở về 0, nhưng bản thân LPPP đã khởi động và đang chạy, và ngay cả khi mất điện tức thời xảy ra ở trạng thái này, HRESET_REQ vẫn có thể được phát ra, nên ở phía PS-CPU, trình tự tuân theo HRESET_REQ.

Xét đến việc xử lý mất điện tức thời ở lần bật nguồn đầu tiên và xử lý mất điện tức thời trong DeepSleep/ERP, dưới đây trình bày trình tự phần mềm của PS-CPU trên S800 khi xảy ra mất điện tức thời.

1. PowerMonitor Xảy ra ngắt Down (mất điện tức thời)
2. "Trạng thái AP_PWR_EN" và "cờ kiểm tra DeepSleep/ERP hay khởi động bật nguồn lần đầu (**g_first_power_on**)" quyết định cách xử lý

AP_PWR_EN=0 và Cờ OFF*: Chuyển sang trình tự tắt nguồn (s800_power_off())

AP_PWR_EN=1 hoặc Cờ ON *: Chờ HRESET_REQ

*Về cờ kiểm tra DeepSleep/ERP hay khởi động bật nguồn lần đầu (**g_first_power_on**) Về cờ này Cờ này được sử dụng vì AP_PWR_EN riêng nó không thể phân biệt được đó là khởi động bật nguồn lần đầu hay DeepSleep/ERP.

Cách sử dụng như sau.

- Khi AP_PWR_EN=1 bật cờ này lên ON
- Khi trình tự tắt nguồn được thực hiện, đặt cờ này về OFF
- ⇒ Ở lần bật nguồn đầu tiên, cờ ở trạng thái OFF
- ⇒ Bất kể AP_PWR_EN, trong DeepSleep/ERP cờ ở trạng thái ON

3. PowerMonitor Chờ PowerMoni phục hồi rồi chuyển sang trình tự khôi phục nguồn

5.1.8. Checksum / Thông tin phiên bản

[Đặc tả checksum]

- MCộng dồn theo thứ tự 20KB dữ liệu của chương trình main từ 0x08000000 đến 0x08005000, và lấy 16 bit thấp của tổng làm checksum.
 - Checksum được tính một lần khi khởi động, bên trong hàm main() g_checksum_value và được lưu vào biến này
 - I2C Có thể lấy giá trị checksum qua địa chỉ 2nd 0x88
- ※5.1.5.3.3 I/O Xem phần mở rộng

[Đặc tả thông tin phiên bản]

- Đổi thông tin phiên bản đang được quản lý dạng 4 byte thành 2 byte
 - 8 bit cao -> phiên bản Major
 - 8 bit thấp -> phiên bản Minor
 - I2C Qua địa chỉ 2nd 0x8C có thể lấy được. 16 bit cao sẽ không được sử dụng
- ※5.1.5.3.3 I/O Xem phần mở rộng
- 4Byte Giao diện và lệnh "EXTEND_VERSION_COMMAND_2" cho phép lấy phiên bản cao hơn của đặc tả sẽ được giữ nguyên
 - MAIN Cùng một đặc tả sẽ được áp dụng cho cả project này và IAP project

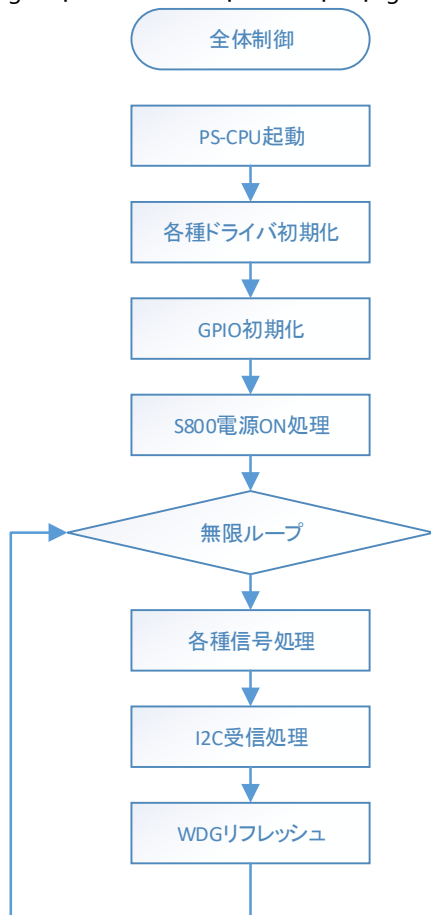
5.2. Sơ đồ tuần tự tổng quan

Sơ đồ tuần tự tổng quan được trình bày dưới đây.

5.2.1. Sơ đồ tuần tự tổng thể

Luồng tổng thể của PS-CPU được trình bày dưới đây.

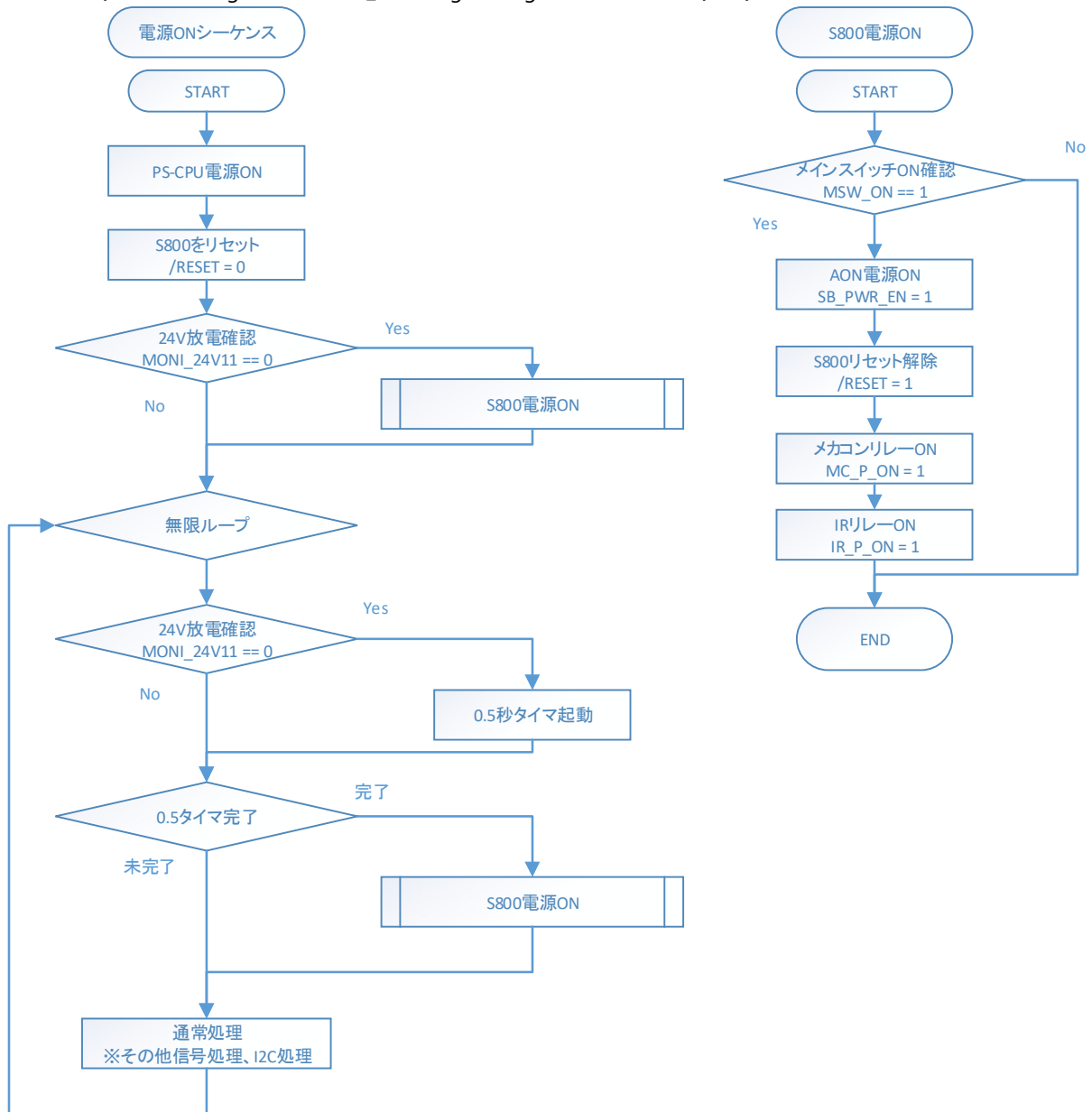
Sau khi PS-CPU khởi động, nó chạy lặp vô hạn, thực hiện xử lý các tín hiệu khác nhau và xử lý nhận I2C, rồi refresh bộ định thời WDG ở cuối vòng lặp. Nếu tại một điểm nào đó trong luồng điều khiển việc xử lý không thể tiếp tục, WDT sẽ không được refresh và một reset tự động sẽ được thực hiện.



5.2.2. Sơ đồ tuần tự bật nguồn

Sơ đồ tuần tự từ khi bật nguồn PS-CPU đến khi bật nguồn cho S800 được trình bày dưới đây.

Sau khi xác nhận đã xả xong 24V và MSW_ON đang ON, nguồn của S800 được bật ON.

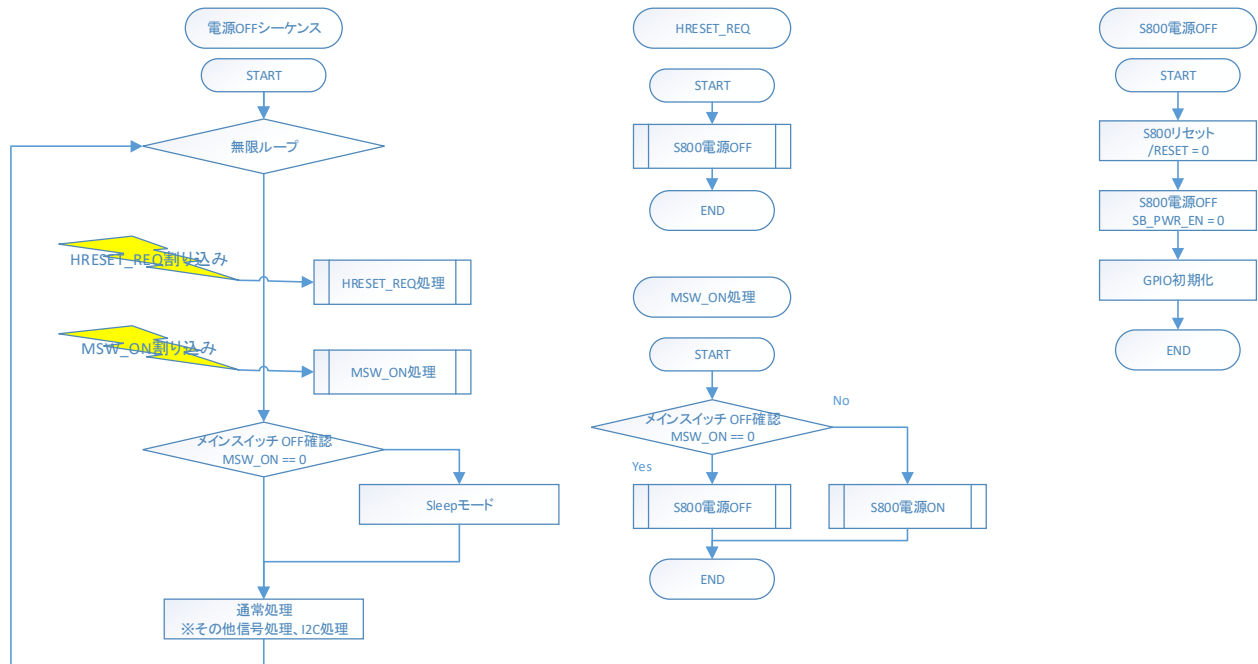


5.2.3. Sơ đồ tuần tự tắt nguồn

Sơ đồ tuần tự tắt nguồn của S800 được trình bày dưới đây.

PS-CPU tắt nguồn của S800 khi HRESET_REQ và MSW_ON xảy ra.

Trong khi MSW_ON ở trạng thái OFF, PS-CPU chuyển sang chế độ Sleep.



5.3. Danh sách hàm

Dưới đây trình bày danh sách các hàm được tạo mới và triển khai trên PS-CPU của S800.

Đối với các driver được Mikawa BSP hỗ trợ, tham khảo tài liệu đặc tả sau.

[¥¥10.128.19.213¥guest¥29P.System IntegrationSoC Development¥PS-CPU¥Doc¥Function Specifications](#)

Function Specifications.docx

[Tài liệu tham khảo]

Tài liệu tham khảo 3. Function Specifications.docx

5.3.1. main project

main/App/entry.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
main	MX xử lý chính của main project	5.9.1.1. main	-
jump2iap	Xử lý chuyển sang IAP project	5.9.1.2. jump2iap	-
Delnit	MX xử lý dừng (shutdown) các driver đang chạy trong main project	5.9.1.3. Delnit	-

main/App/km_extend_io.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
change_cmd2array	Chuyển địa chỉ 2nd thành chỉ số (index) của biến nội bộ	5.9.1.4. change_cmd2array	-
io_extend_read	Đọc từ các port dùng cho I/O mở rộng	5.9.1.5. io_extend_read	-
io_extend_write	Ghi vào các port dùng cho I/O mở rộng	5.9.1.6. io_extend_write	-
io_extend_memory_write	Lưu trạng thái của các port I/O mở rộng vào bộ nhớ nội bộ	5.9.1.7. io_extend_memory_write	-
internal_sts_init	Khởi tạo trạng thái nội bộ	5.9.1.8. internal_sts_init	-
mc3_3von_moni_proc	Xử lý tín hiệu MC3.3VON_MONI	5.9.1.9. mc3_3von_moni_proc	-
power_monitor_proc	Xử lý tín hiệu POWER_MONITOR	5.9.1.10. power_monitor_proc	-
hreset_req_proc	Xử lý tín hiệu HRESET_REQ	5.9.1.11. hreset_req_proc	-
extend_output_init	Khởi tạo phần mở rộng đầu ra	5.9.1.12. extend_output_init	-
s800_power_off	Xử lý tắt nguồn S800	5.9.1.13. s800_power_off	-
moni_24v_proc	Xử lý xác nhận xả 24V	5.9.1.14. moni_24v_proc	-
s800_power_on	Xử lý bật nguồn S800	5.9.1.15. s800_power_on	-
ap_power_en_proc	Xử lý tín hiệu AP_PWR_EN	5.9.1.16. ap_power_en_proc	-
erp_sensor_proc	Xử lý tín hiệu ERP_SENSOR_ON	5.9.1.17. erp_sensor_proc	-
rst_usb_proc	Xử lý tín hiệu RST_USB	5.9.1.18. rst_usb_proc	-
mc_p_on_proc	Xử lý tín hiệu MC_P_ON	5.9.1.19. mc_p_on_proc	-
ir_p_on_proc	Xử lý tín hiệu IR_P_ON	5.9.1.20. ir_p_on_proc	-
mc_oe_proc	Xử lý tín hiệu MC_OE	5.9.1.21. mc_oe_proc	-
mc_pwr_en_proc	Xử lý tín hiệu MC_PWR_EN	5.9.1.22. mc_pwr_en_proc	-
msw_on_proc	Xử lý tín hiệu MSW_ON	5.9.1.23. msw_on_proc	-

main/App/km_i2c.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
rx_comp	Xử lý hoàn tất dữ liệu nhận I2C	5.9.1.24. rx_comp	-
change_state	Xử lý chuyển đổi gửi/nhận của I2C slave	5.9.1.25. change_state	-
i2c_recv_first	Xử lý nhận I2C ban đầu	5.9.1.26. i2c_recv_first	-
i2c_recv_wait	Xử lý nhận I2C	5.9.1.27. i2c_recv_wait	-

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

checksum_calc	Tính checksum của chương trình main	5.9.1.28. checksum_calc	-
---------------	-------------------------------------	-------------------------	---

main/App/km_it.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
hreset_req_proc_handler	Handler của /HRESET_REQ	5.9.1.29. hreset_req_proc_handler	-
km_exti_callback	Handler ngắt ngoài	5.9.1.30. km_exti_callback	-
km_it_init	Xử lý khởi tạo ngắt tùy chỉnh của KM	5.9.1.31. km_it_init	
HAL_TIM_PeriodElapsedCallback	Handler của timer (chu kỳ 1 ms)	5.9.1.32. HAL_TIM_PeriodElapsedCallback	-
km_timer_set	Thiết lập timer	5.9.1.33. km_timer_set	-
km_timer_get_state	Lấy trạng thái timer	5.9.1.34. km_timer_get_state	-

main/App/km_adc.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
adc_moni_5v_start_proc	Xử lý phát hiện 5V	5.9.1.35. adc_moni_5v_start_proc	-
adc_moni_5v_off_proc	Xử lý phát hiện sụt 5V	5.9.1.36. adc_moni_5v_off_proc	-

common/Drivers/BSP/Src/BSP_PWR_STM32F03x_Nucleo.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
BSP_PWR_enter_stopmode	Hàm chuyển sang chế độ STOP	5.9.1.37.BSP_PWR_enter_stopmode	-
BSP_PWR_enter_sleepmode	Hàm chuyển sang chế độ Sleep	5.9.1.38.BSP_PWR_enter_sleepmode	-

5.3.2. iap project

iap/App/entry_iap.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
main	MXử lý chính của main project	5.9.2.1. main	-

iap/App/km_i2c_iap.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
rx_comp	IXử lý hoàn tất dữ liệu nhận I2C	5.9.2.2. rx_comp	-
change_state	Xử lý chuyển đổi gửi/nhận của I2C slave	5.9.2.3. change_state	-
i2c_recv_first	Xử lý nhận I2C ban đầu	5.9.2.4. i2c_recv_first	-
i2c_recv_wait	Xử lý nhận I2C	5.9.2.5. i2c_recv_wait	-
flash_write_32	Ghi flash cho chương trình main	5.9.2.6. flash_write_32	-
main_project_flash_erase	Hàm xóa flash của chương trình main	5.9.2.7. main_project_flash_erase	-
check_sum_calc	Hàm kiểm tra checksum dữ liệu chương trình main	5.9.2.8. check_sum_calc	-

iap/App/km_extend_io_iap.c

Tên hàm	Tổng quan xử lý	Điều khiển liên quan	Ghi chú
msw_on_proc_iap	MSW_ONXử lý tín hiệu	5.9.2.9. msw_on_proc_iap	-
km_exti_callback_iap	Xử lý handler ngắt ngoài	5.9.2.10. km_exti_callback_iap	-
km_it_init_iap	Xử lý khởi tạo ngắt	5.9.2.11. km_it_init_iap	-

5.4. Đặc tả hằng số

Danh sách các hằng số được bổ sung mới trên PS-CPU của S800.

5.4.1. main project

main/Inc/entry.h

Tên hằng số	Giá trị	Nội dung
IAP_DEFAULT_ADD	0x08005000	Địa chỉ Flash ROM nơi đặt IAP project

main/Inc/km_extend_io.h

Tên hằng số	Giá trị	Nội dung
BIT_CALC(x)	1 << x	Macro dùng để dịch bit
OUTPUT_EXTEND_IR_P_ON_BIT	0	Vị trí bit của IR_P_ON tại địa chỉ 2nd 0x70 trong truyền thông I2C
OUTPUT_EXTEND_MC_P_ON_BIT	5	Vị trí bit của MC_P_ON tại địa chỉ 2nd 0x70 trong truyền thông I2C
OUTPUT_EXTEND_RESET_BIT	2	Vị trí bit của /RESET tại địa chỉ 2nd 0x70 trong truyền thông I2C
OUTPUT_EXTEND_SB_PWR_EN_BIT	3	Vị trí bit của SB_PWR_EN tại địa chỉ 2nd 0x70 trong truyền thông I2C
OUTPUT_EXTEND_WAKEUP_BIT	0	Vị trí bit của WAKEUP tại địa chỉ 2nd 0x60 trong truyền thông I2C
OUTPUT_EXTEND_ERP_SENSOR_ON_BIT	1	Vị trí bit của ERP_SEMSOR_ON tại địa chỉ 2nd 0x70 trong truyền thông I2C
OUTPUT_EXTEND_RST_SLP2_BIT	9	Vị trí bit của /RST_SLP2 tại địa chỉ 2nd 0x70 trong truyền thông I2C
INPUT_EXTEND_24V11_MONI_BIT	8	Vị trí bit của 5V_MONI tại địa chỉ 2nd 0x74 trong truyền thông I2C
INPUT_EXTEND_MC_3.3V_ON_MONI_BIT	15	Vị trí bit của MC_3.3V_ON_MONI tại địa chỉ 2nd 0x74 trong truyền thông I2C
INPUT_EXTEND_POWER_MONITOR_BIT	4	Vị trí bit của POWER_MONITOR tại địa chỉ 2nd 0x74 trong truyền thông I2C
INPUT_EXTEND_HRESET_REQ_BIT	1	Vị trí bit của HRESET_REQ tại địa chỉ 2nd 0x74 trong truyền thông I2C
INPUT_EXTEND_AP_PWR_EN_BIT	2	Vị trí bit của AP_PWR_EN tại địa chỉ 2nd 0x74 trong truyền

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

		thông I2C
INPUT_EXTEND_MSW_ON_BIT	10	Vị trí bit của MSW_ON tại địa chỉ 2nd 0x74 trong truyền thông I2C MSW_ON tại địa chỉ 2nd 0x70 trong truyền thông I2C
MC_P_ON_NUM	TYPE_Output_Extend 0	Số chỉ mục nơi lưu MC_P_ON
IR_P_ON_NUM	TYPE_Output_Extend 0	Số chỉ mục nơi lưu IR_P_ON
ERP_SENSOR_ON_NUM	TYPE_Output_Extend 0	Số I/O mở rộng của ERP_SENSOR_ON
RST_SLP2_NUM	TYPE_Output_Extend 0	Số I/O mở rộng của RST_SLP2
STATE_INT	0	Gọi từ ngắt
STATE_NORMAL	1	Gọi từ trạng thái bình thường
POWER_ON_FLG_START	0	Trạng thái bật nguồn hoặc khởi động lại
POWER_ON_FLG_NORMAL	1	Trạng thái sau khi khởi động
FIRST_POWER_ON_FLG_START	0	Khởi động bật nguồn lần đầu (dùng cho mất điện tức thời)
FIRST_POWER_ON_FLG_NORMAL	1	Không phải khởi động bật nguồn lần đầu (dùng cho mất điện tức thời)

main/Inc/km_i2c.h

Tên hằng số	Giá trị	Nội dung
COMMAND_RWC_BUF_SIZE	0x10	Kích thước buffer nhận lệnh Read/Write
COMMAND_TX_SIZE	2	Kích thước truyền
READ_CMD_SIZE	1	Kích thước buffer nhận lệnh Read
SECOND_ADDR_ARRAY	0x0	2 nd Vị trí lưu địa chỉ
DATA_ARRAY	0x1	Vị trí lưu dữ liệu
DATA_ARRAY_SIZE	0x4	Kích thước dữ liệu
RTC_TR_CMD	0x0	Thanh ghi TR – địa chỉ 2 nd (định nghĩa)
RTC_TR_CMD_2	0x2	Thanh ghi TR – địa chỉ 2 nd (định nghĩa)
RTC_DR_CMD	0x4	Thanh ghi DR – địa chỉ 2 nd (định nghĩa)
RTC_DR_CMD_2	0x6	Thanh ghi DR – địa chỉ 2 nd (định nghĩa)
RTC_CR_CMD	0x8	Thanh ghi CR – địa chỉ 2 nd (định nghĩa)
RTC_CR_CMD_2	0xA	Thanh ghi CR – địa chỉ 2 nd (định nghĩa)
RTC_ISR_CMD	0xC	Thanh ghi ISR – địa chỉ 2 nd (định nghĩa)
RTC_ISR_CMD_2	0xE	Thanh ghi ISR – địa chỉ 2 nd (định nghĩa)
RTC_PRER_CMD	0x10	Thanh ghi PRER – địa chỉ 2 nd (định nghĩa)
RTC_PRER_CMD_2	0x12	Thanh ghi PRER – địa chỉ 2 nd (định nghĩa)
RTC_WUTR_CMD	0x14	Thanh ghi WUTR – địa chỉ 2 nd (định nghĩa)
RTC_WUTR_CMD_2	0x16	Thanh ghi WUTR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMAR_CMD	0x1C	Thanh ghi ALRMAR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMAR_CMD_2	0x1E	Thanh ghi ALRMAR – địa chỉ 2 nd (định nghĩa)
RTC_WPR_CMD	0x24	Thanh ghi WPR – địa chỉ 2 nd (định nghĩa)
RTC_WPR_CMD_2	0x26	Thanh ghi WPR – địa chỉ 2 nd (định nghĩa)
RTC_SSR_CMD	0x28	Thanh ghi SSR – địa chỉ 2 nd (định nghĩa)
RTC_SSR_CMD_2	0x2A	Thanh ghi SSR – địa chỉ 2 nd (định nghĩa)
RTC_SHIFTR_CMD	0x2C	Thanh ghi SHIFTR – địa chỉ 2 nd (định nghĩa)
RTC_SHIFTR_CMD_2	0x2E	Thanh ghi SHIFTR – địa chỉ 2 nd (định nghĩa)
RTC_TSTR_CMD	0x30	Thanh ghi TSTR – địa chỉ 2 nd (định nghĩa)
RTC_TSTR_CMD_2	0x32	Thanh ghi TSTR – địa chỉ 2 nd (định nghĩa)
RTC_TSDR_CMD	0x34	Thanh ghi TSDR – địa chỉ 2 nd (định nghĩa)
RTC_TSDR_CMD_2	0x36	Thanh ghi TSDR – địa chỉ 2 nd (định nghĩa)
RTC_TSSSR_CMD	0x38	Thanh ghi TSSSR – địa chỉ 2 nd (định nghĩa)
RTC_TSSSR_CMD_2	0x3A	Thanh ghi TSSSR – địa chỉ 2 nd (định nghĩa)
RTC_CALR_CMD	0x3C	Thanh ghi CALR – địa chỉ 2 nd (định nghĩa)
RTC_CALR_CMD_2	0x3E	Thanh ghi CALR – địa chỉ 2 nd (định nghĩa)
RTC_TAFCR_CMD	0x40	Thanh ghi TAFCR – địa chỉ 2 nd (định nghĩa)
RTC_TAFCR_CMD_2	0x42	Thanh ghi TAFCR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMASSR_CMD	0x44	Thanh ghi ALRMASSR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMASSR_CMD_2	0x46	Thanh ghi ALRMASSR – địa chỉ 2 nd (định nghĩa)

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

RTC_BKP0R_CMD	0x50	BKPOR0 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP0R_CMD_2	0x52	BKPOR0 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP1R_CMD	0x54	BKPOR1 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP1R_CMD_2	0x56	BKPOR1 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP2R_CMD	0x58	BKPOR2 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP2R_CMD_2	0x5A	BKPOR2 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP3R_CMD	0x5C	BKPOR3 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP3R_CMD_2	0x5E	BKPOR3 – địa chỉ thanh ghi 2 nd (định nghĩa)
INT_STS_COMMAND	0x60	Trạng thái ngắt – địa chỉ 2 nd (định nghĩa)
INT_STS_COMMAND_2	0x62	Trạng thái ngắt – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND	0x68	Trạng thái nội bộ – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND_2	0x6E	Trạng thái nội bộ – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND_BOO T_STATES	0x00000001	Vị trí bit của trạng thái khởi động
INTERNEAL_STS_COMMAND_BOO T_STATES_COLD	0x0	Coldboot
INTERNEAL_STS_COMMAND_BOO T_STATES_REBOOT	0x1	Khởi động lại (Reboot)
EXTEND_OUTPUT_READ_COMMA ND	0x70	Thanh ghi Read của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_READ_COMMA ND_2	0x72	Thanh ghi Read của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_HIGH_COMMA ND	0x74	Thanh ghi thiết lập Bit High của đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_HIGH_COMMA ND_2	0x76	Thanh ghi thiết lập Bit High của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_LOW_COMMAN D	0x78	Thanh ghi thiết lập Bit Low của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_LOW_COMMAN D_2	0x7A	Thanh ghi thiết lập Bit Low của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_INPUT_COMMAND	0x80	Thanh ghi đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_INPUT_COMMAND_2	0x82	Thanh ghi đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_IO_COMMAND_BASE	EXTEND_OUTPUT_C OMMAND	Địa chỉ cơ sở của các lệnh IO mở rộng
EXTEND_VERSION_COMMAND	0x8C	Thông tin phiên bản – địa chỉ 2 nd (định nghĩa)
EXTEND_VERSION_COMMAND_2	0x8E	Thông tin phiên bản – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND	0x90	Yêu cầu ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_INF O	0x94	Hoàn tất ghi lại dữ liệu chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_INF O_COMP	0x00000004	Hoàn tất ghi lại dữ liệu chương trình – bit hoàn tất dữ liệu chương trình
IAP_PG_COMMAND_CHANGE_INF O_CHECKSUM	0x0000FF00	Hoàn tất ghi lại dữ liệu chương trình – bit checksum
IAP_PG_COMMAND_CHANGE_INF O_CHECKSUM_SHIFT	16	Hoàn tất ghi lại dữ liệu chương trình – giá trị dịch bit checksum
IAP_PG_COMMAND_CHANGE_STA TUS	0x98	Trạng thái ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_STA TUS_IAP	0x00000001	Trạng thái ghi lại chương trình – trạng thái chuyển sang IAP
IAP_PG_COMMAND_CHANGE_STA TUS_DATA_CMD_STS	0x00000002	Trạng thái ghi lại chương trình – trạng thái dữ liệu ghi lại chương trình
IAP_PG_COMMAND_CHANGE_STA TUS_PG_CMD_STS	0x00000004	Trạng thái ghi lại chương trình – trạng thái hoàn tất dữ liệu chương trình
IAP_PG_COMMAND_CHANGE_STA TUS_CHKSUM_ERR	0x00010000	Trạng thái ghi lại chương trình – lỗi checksum
IAP_PG_COMMAND_CHANGE_STA TUS_PSCPU_ERR	0x00020000	Trạng thái ghi lại chương trình – lỗi nội bộ PS-CPU
IAP_PG_COMMAND_DATA	0xA0	Dữ liệu ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_OFFSET	IAP_PG_COMMAND	Offset của yêu cầu ghi lại chương trình

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

IAP_PG_COMMAND_NUM	4	Số lượng lệnh ghi lại chương trình
IAP_REG_INDEX_CALC(x)	((x - IAP_PG_COMMAND_OFFSET)/sizeof(uint32_t))	Macro tính chỉ số thanh ghi IAP
IAP_PG_COMMAND_MAGICNUM	0x11223344	Magic number cho yêu cầu ghi lại chương trình
RTC_WRITEPROTECT_DISABLE	Xem chú thích *1	Vô hiệu hóa bảo vệ ghi của các thanh ghi RTC
RTC_WRITEPROTECT_ENABLE	Xem chú thích 2	Kích hoạt bảo vệ ghi của các thanh ghi RTC
MAIN_APPLICATION_ADDRESS	(uint32_t)0x08000000	Địa chỉ bắt đầu của MAIN project
MAIN_APPLICATION_SIZE	(uint32_t)0x5000	Kích thước của MAIN project

※1

```
#define RTC_WRITEPROTECT_DISABLE
do{
    __IO uint32_t *w_p_addr=(uint32_t*)(RTC_BASE + RTC_WRITEPROTECT_COMMAND);
    *w_p_addr = 0xCA;
    *w_p_addr = 0x53;
}while(0)
```

※2

```
#define RTC_WRITEPROTECT_ENABLE
do{
    uint32_t *w_p_addr=(uint32_t*)(RTC_BASE + RTC_WRITEPROTECT_COMMAND);
    *w_p_addr = 0xFF;
}while(0)
```

main/Inc/km_it.h

Tên hằng số	Giá trị	Nội dung
KM_TIMER_MAX_TIME	32768	Giá trị timer lớn nhất 32.768s

main/Inc/km_adc.h

Tên hằng số	Giá trị	Nội dung
MONI_5V_ADC_CH	ADC_CH_0	Kênh ADC cho 5V
ADC_VAL_POWER_ON	0xb84	$4.75V(4.75 / 2 * (2^{12} - 1) / 3.3 \div 0xb84)$
ADC_VAL_POWER_OFF	0xACA	$4.45V(4.45 / 2 * (2^{12} - 1) / 3.3 \div 0xACA)$

main/Inc/km_ver.h

Tên hằng số	Giá trị	Nội dung
VERSION_TYPE		Thông tin phiên bản
VERSION_TYPE_SHIFT_16BIT		Giá trị dịch để hiển thị thông tin phiên bản dạng 16-bit
VERSION_TYPE_SHIFT_32BIT		Giá trị dịch để hiển thị thông tin phiên bản dạng 32-bit
REVISION_TYPE		Thông tin revision
REVISION_TYPE_SHIFT_16BIT		Giá trị dịch để hiển thị thông tin revision dạng 16-bit
REVISION_TYPE_SHIFT_32BIT		Giá trị dịch để hiển thị thông tin revision dạng 32-bit
MODEL_TYPE		Thông tin Model
MODEL_TYPE_SHIFT_16BIT		Giá trị dịch để hiển thị thông tin Model dạng 16-bit
MODEL_TYPE_SHIFT_32BIT		Giá trị dịch để hiển thị thông tin Model dạng 32-bit
VERSION_MAIN_U		Thông tin phiên bản – phần cao (16-bit)
VERSION_MAIN_L		Thông tin phiên bản – phần thấp (16-bit)
VERSION_MAIN		Thông tin phiên bản (32-bit)
VERSION_MAJOR	0x01	Major Version – thông tin
VERSION_MINOR	0x00	Minor Version – thông tin
VERSION_TYPE_SHIFT_16BIT	8	Giá trị dịch của phiên bản 16bit
VERSION_MAIN_U	((VERSION_MAJOR << VERSION_TYPE_SHIFT_16BIT) VERSION_MINOR)	Thông tin phiên bản (16 bit)
VERSION_MAIN_L	0	Thông tin phiên bản (không sử dụng)

VERSION_MAIN	VERSION_MAIN_U	Thông tin phiên bản (16 bit)
--------------	----------------	------------------------------

5. 4. 2. iap project

iap/Inc/km_i2c_iap.h

Tên hằng số	Giá trị	Nội dung
COMMAND_RWC_BUF_SIZE	1025	Kích thước buffer nhận lệnh Read/Write
COMMAND_TX_SIZE	0x2	Kích thước buffer gửi cho lệnh Read
SECOND_ADDR_ARRAY	0x0	2 nd Vị trí lưu địa chỉ
DATA_ARRAY	0x1	Vị trí lưu dữ liệu
DATA_ARRAY_SIZE	0x4	Kích thước dữ liệu
RTC_TR_CMD	0x0	Thanh ghi TR – địa chỉ 2 nd (định nghĩa)
RTC_TR_CMD_2	0x2	Thanh ghi TR – địa chỉ 2 nd (định nghĩa)
RTC_DR_CMD	0x4	Thanh ghi DR – địa chỉ 2 nd (định nghĩa)
RTC_DR_CMD_2	0x6	Thanh ghi DR – địa chỉ 2 nd (định nghĩa)
RTC_CR_CMD	0x8	Thanh ghi CR – địa chỉ 2 nd (định nghĩa)
RTC_CR_CMD_2	0xA	Thanh ghi CR – địa chỉ 2 nd (định nghĩa)
RTC_ISR_CMD	0xC	Thanh ghi ISR – địa chỉ 2 nd (định nghĩa)
RTC_ISR_CMD_2	0xE	Thanh ghi ISR – địa chỉ 2 nd (định nghĩa)
RTC_PRER_CMD	0x10	Thanh ghi PRER – địa chỉ 2 nd (định nghĩa)
RTC_PRER_CMD_2	0x12	Thanh ghi PRER – địa chỉ 2 nd (định nghĩa)
RTC_WUTR_CMD	0x14	Thanh ghi WUTR – địa chỉ 2 nd (định nghĩa)
RTC_WUTR_CMD_2	0x16	Thanh ghi WUTR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMAR_CMD	0x1C	Thanh ghi ALRMAR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMAR_CMD_2	0x1E	Thanh ghi ALRMAR – địa chỉ 2 nd (định nghĩa)
RTC_WPR_CMD	0x24	Thanh ghi WPR – địa chỉ 2 nd (định nghĩa)
RTC_WPR_CMD_2	0x26	Thanh ghi WPR – địa chỉ 2 nd (định nghĩa)
RTC_SSR_CMD	0x28	Thanh ghi SSR – địa chỉ 2 nd (định nghĩa)
RTC_SSR_CMD_2	0x2A	Thanh ghi SSR – địa chỉ 2 nd (định nghĩa)
RTC_SHIFTR_CMD	0x2C	Thanh ghi SHIFTR – địa chỉ 2 nd (định nghĩa)
RTC_SHIFTR_CMD_2	0x2E	Thanh ghi SHIFTR – địa chỉ 2 nd (định nghĩa)
RTC_TSTR_CMD	0x30	Thanh ghi TSTR – địa chỉ 2 nd (định nghĩa)
RTC_TSTR_CMD_2	0x32	Thanh ghi TSTR – địa chỉ 2 nd (định nghĩa)
RTC_TSDR_CMD	0x34	Thanh ghi TSDR – địa chỉ 2 nd (định nghĩa)
RTC_TSDR_CMD_2	0x36	Thanh ghi TSDR – địa chỉ 2 nd (định nghĩa)
RTC_TSSSR_CMD	0x38	Thanh ghi TSSSR – địa chỉ 2 nd (định nghĩa)
RTC_TSSSR_CMD_2	0x3A	Thanh ghi TSSSR – địa chỉ 2 nd (định nghĩa)
RTC_CALR_CMD	0x3C	Thanh ghi CALR – địa chỉ 2 nd (định nghĩa)
RTC_CALR_CMD_2	0x3E	Thanh ghi CALR – địa chỉ 2 nd (định nghĩa)
RTC_TAFCR_CMD	0x40	Thanh ghi TAFCR – địa chỉ 2 nd (định nghĩa)
RTC_TAFCR_CMD_2	0x42	Thanh ghi TAFCR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMASR_CMD	0x44	Thanh ghi ALRMASR – địa chỉ 2 nd (định nghĩa)
RTC_ALRMASR_CMD_2	0x46	Thanh ghi ALRMASR – địa chỉ 2 nd (định nghĩa)
RTC_BKP0R_CMD	0x50	BKPOR0 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP0R_CMD_2	0x52	BKPOR0 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP1R_CMD	0x54	BKPOR1 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP1R_CMD_2	0x56	BKPOR1 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP2R_CMD	0x58	BKPOR2 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP2R_CMD_2	0x5A	BKPOR2 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP3R_CMD	0x5C	BKPOR3 – địa chỉ thanh ghi 2 nd (định nghĩa)
RTC_BKP3R_CMD_2	0x5E	BKPOR3 – địa chỉ thanh ghi 2 nd (định nghĩa)
INT_STS_COMMAND	0x60	Trạng thái ngắt – địa chỉ 2 nd (định nghĩa)
INT_STS_COMMAND_2	0x62	Trạng thái ngắt – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND	0x68	Trạng thái nội bộ – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND_2	0x6E	Trạng thái nội bộ – địa chỉ 2 nd (định nghĩa)
INTERNEAL_STS_COMMAND_BOOT_STATES	0x00000001	Vị trí bit của trạng thái khởi động
INTERNEAL_STS_COMMAND_BOOT_STATES_COLD	0x0	Coldboot
INTERNEAL_STS_COMMAND_BOOT_STATES_WARM	0x1	Khởi động lại (Reboot)

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

T_STATES_REBOOT		
EXTEND_OUTPUT_READ_COMMAND	0x70	Thanh ghi Read của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_READ_COMMAND_2	0x72	Thanh ghi Read của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_HIGH_COMMAND	0x74	Thanh ghi thiết lập Bit High của đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_HIGH_COMMAND_2	0x76	Thanh ghi thiết lập Bit High của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_LOW_COMMAND	0x78	Thanh ghi thiết lập Bit Low của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_OUTPUT_LOW_COMMAND_2	0x7A	Thanh ghi thiết lập Bit Low của đầu ra mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_INPUT_COMMAND	0x80	Thanh ghi đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_INPUT_COMMAND_2	0x82	Thanh ghi đầu vào mở rộng – địa chỉ 2 nd (định nghĩa)
EXTEND_IO_COMMAND_BASE	EXTEND_OUTPUT_COMMAND	Địa chỉ cơ sở của các lệnh IO mở rộng
EXTEND_VERSION_COMMAND	0x8C	Thông tin phiên bản – địa chỉ 2 nd (định nghĩa)
EXTEND_VERSION_COMMAND_2	0x8E	Thông tin phiên bản – địa chỉ 2 nd (định nghĩa)
IAP_COMMAND	0x90	Yêu cầu ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_2	0x92	Yêu cầu ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_INFO	0x94	Hoàn tất ghi lại dữ liệu chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_INFO_2	0x96	Hoàn tất ghi lại dữ liệu chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_INFO_COMP	0x00000004	Hoàn tất ghi lại dữ liệu chương trình – bit hoàn tất dữ liệu chương trình
IAP_PG_COMMAND_CHANGE_INFO_CHECKSUM	0x0000FF00	Hoàn tất ghi lại dữ liệu chương trình – bit checksum
IAP_PG_COMMAND_CHANGE_INFO_CHECKSUM_SHIFT	8	Hoàn tất ghi lại dữ liệu chương trình – giá trị dịch bit checksum
IAP_PG_COMMAND_CHANGE_STATUS	0x98	Trạng thái ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_STATUS_2	0x9A	Trạng thái ghi lại chương trình – địa chỉ 2 nd (định nghĩa)
IAP_PG_COMMAND_CHANGE_STATUS_IAP	0x00000001	Trạng thái ghi lại chương trình – trạng thái chuyển sang IAP
IAP_PG_COMMAND_CHANGE_STATUS_DATA_CMD_STS	0x00000002	Trạng thái ghi lại chương trình – trạng thái dữ liệu ghi lại chương trình
IAP_PG_COMMAND_CHANGE_STATUS_PG_CMD_STS	0x00000004	Trạng thái ghi lại chương trình – trạng thái hoàn tất dữ liệu chương trình
IAP_PG_COMMAND_CHANGE_STATUS_CHKSUM_ERR	0x00010000	Trạng thái ghi lại chương trình – lỗi checksum
IAP_PG_COMMAND_CHANGE_STATUS_PSCPU_ERR	0x00020000	Trạng thái ghi lại chương trình – lỗi nội bộ PS-CPU
IAP_PG_DATA_COMMAND	0xA0	2 nd Định nghĩa địa chỉ: 0xA0
IAP_PG_COMMAND_OFFSET	IAP_PG_COMMAND	Yêu cầu ghi lại chương trình
IAP_PG_COMMAND_NUM	4	Số lượng lệnh ghi lại chương trình
IAP_REG_INDEX_CALC(x)	((x - IAP_PG_COMMAND_OFFSET)/sizeof(uint32_t))	Macro tính chỉ số thanh ghi IAP
IAP_PG_COMMAND_MAGICNUM	0x11223344	Magic number cho yêu cầu ghi lại chương trình
RTC_WRITEPROTECT_DISABLE	Xem chú thích *1	Vô hiệu hóa bảo vệ ghi của các thanh ghi RTC
RTC_WRITEPROTECT_ENABLE	Xem chú thích 2	Kích hoạt bảo vệ ghi của các thanh ghi RTC

※1

#define RTC_WRITEPROTECT_DISABLE	
do{	¥
__IO uint32_t *w_p_addr=(uint32_t*)(RTC_BASE + RTC_WRITEPROTECT_COMMAND);	¥

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

*w_p_addr = 0xCA;	¥
*w_p_addr = 0x53;	¥
}while(0)	¥
※2	
#define RTC_WRITEPROTECT_ENABLE	
do{	¥
uint32_t *w_p_addr=(uint32_t*)(RTC_BASE + RTC_WRITEPROTECT_COMMAND);	¥
*w_p_addr = 0xFF;	¥
}while(0)	

iap/App/km_i2c_iap.c

Tên hằng số	Giá trị	Nội dung
MAIN_APPLICATION_ADDRESS	(uint32_t)0x08000000	Địa chỉ bắt đầu của vùng flash ROM lưu Main project
MAIN_APPLICATION_SIZE	(uint32_t)0x4000	Kích thước vùng flash ROM lưu Main project

5. 5. Đặc tả kiểu liệt kê (enum)

Danh sách các kiểu liệt kê (enum) được bổ sung mới trên PS-CPU của S800.

5. 5. 1. main project

main/Inc/km_extend_io.h

Tên kiểu liệt kê	TYPE_Io_Extend_Num	
Tên hằng số	Giá trị	Nội dung
TYPE_Output_Extend0	0	Số của đầu ra mở rộng
TYPE_Input_Extend0	1	Số của đầu vào mở rộng
TYPE_Int_Sts0	2	Số của trạng thái ngắt
TYPE_Internal_Sts0	3	Số của trạng thái nội bộ
TYPE_Io_Extend_Max	4	Giá trị lớn nhất của số I/O mở rộng

main/App/km_i2c.c

Tên kiểu liệt kê	-	
Tên hằng số	Giá trị	Nội dung
TYPE_State_Init	0	Trạng thái I2C – trạng thái khởi tạo
TYPE_State_Wait_Write	1	Trạng thái I2C – trạng thái Write
TYPE_State_Wait_Read	2	Trạng thái I2C – trạng thái Read

main/Inc/km_it.h

Tên kiểu liệt kê	TYPE_Km_Timer_State-	
Tên hằng số	Giá trị	Nội dung
TYPE_Km_Timer_Normal	0	Trạng thái Timer – trạng thái khởi tạo
TYPE_Km_Timer_Start	1	Trạng thái Timer – trạng thái Start
TYPE_Km_Timer_End	2	Trạng thái Timer – trạng thái kết thúc
TYPE_KM_Timer_Max	3	Giá trị lớn nhất của trạng thái timer

main/Inc/km_it.h

Tên kiểu liệt kê	TYPE_Km_Timer_Kind	
Tên hằng số	Giá trị	Nội dung
TYPE_Km_Timer_Rstusb	0	Timer dùng cho /RST_USB
TYPE_Km_Timer_MONI_24V11	1	Timer kiểm tra xả 24V
TYPE_Km_Timer_Kind_Max	2	Giá trị lớn nhất của loại timer

5. 5. 2. iap project

iap/App/km_i2c_iap.c

Tên kiểu liệt kê	-	
Tên hằng số	Giá trị	Nội dung
TYPE_State_Init	0	Trạng thái I2C – trạng thái khởi tạo
TYPE_State_Wait_Write	1	Trạng thái I2C – trạng thái Write
TYPE_State_Wait_Read	2	Trạng thái I2C – trạng thái Read

5. 6. Đặc tả cấu trúc (struct)

Danh sách các cấu trúc (struct) được bổ sung mới trên PS-CPU của S800.

5. 6. 1. main project

main/Inc/km_extend_io.h

Tên cấu trúc	km_io	
Tên thành viên	Kiểu	Nội dung
extend_num	uint32_t	Số I/O mở rộng
bit	uint32_t	Vị trí bit của I/O mở rộng
port	GPIO_TypeDef *	Kiểu port của I/O mở rộng
pin	uint16_t	Vị trí chân của I/O mở rộng

main/App/km_i2c.c

Tên cấu trúc	_i2c_info	
Tên thành viên	Kiểu	Nội dung
state	int	Trạng thái Read/Write
rx_buf[COMMAND_RWC_BUF_SIZE]	uint8_t	Buffer nhận
second_addr	uint8_t	Địa chỉ thứ 2 (2nd address)
write_comp	int	Cờ hoàn tất Write
read_comp	int	Cờ hoàn tất Read
xfer_size	int	Kích thước truyền

main/Inc/km_it.h

Tên cấu trúc	km_timer_st	
Tên thành viên	Kiểu	Nội dung
state	TYPE_Km_Timer_State	Trạng thái timer
ms_time	MillsecType	Giá trị timer tính bằng ms (MAX=32768ms)

5. 6. 1. iap project

iap/App/km_i2c_iap.c

Tên cấu trúc	_i2c_info	
Tên thành viên	Kiểu	Nội dung
state	int	Trạng thái Read/Write
rx_buf[COMMAND_RWC_BUF_SIZE]	uint8_t	Buffer nhận
second_addr	uint8_t	Địa chỉ thứ 2 (2nd address)
write_comp	int	Cờ hoàn tất Write
read_comp	int	Cờ hoàn tất Read
xfer_size	int	Kích thước truyền

5. 7. Biến toàn cục

Danh sách các biến toàn cục được bổ sung mới trên PS-CPU của S800.

5. 7. 1. main project

iap/App/km_extend_io.c

uint32_t g_io_extend_memory[TYPE_Io_Extend_Max];
--

Biến dùng để lưu nội dung I/O mở rộng trong bộ nhớ.

iap/App/km_extend_io.c

int32_t g_power_on_flg = POWER_ON_FLG_START;
--

Cờ giữ trạng thái khởi động.

iap/App/km_extend_io.c

int32_t g_24v_check_flg = STATE_NONACTIVE;
--

Cờ cho biết có cần kiểm tra việc xả 24V hay không.

iap/App/km_extend_io.c

g_first_power_on = FIRST_POWER_ON_FLG_START;
--

Cờ để xác định đây có phải là khởi động bật nguồn lần đầu hay không, dùng cho việc kiểm tra mất điện tức thời

main/App/km_i2c.c

struct _i2c_info i2c_info;

Biến quản lý trạng thái I2C.

main/App/km_i2c.c

uint32_t tx_data=0;

Buffer gửi của I2C.

main/App/km_i2c.c

uint16_t g_checksum_value;

Checksum.

5. 7. 2. iap project

iap/App/km_i2c_iap.c

struct __i2c_info i2c_info;

Biến quản lý trạng thái I2C.

iap/App/km_i2c_iap.c

static uint32_t dest_addr;

Địa chỉ flash cần ghi lại.

iap/App/km_i2c_iap.c

uint32_t g_iap_reg[IAP_PG_COMMAND_NUM];

IAP Các thanh ghi I2C.

5. 8. Danh sách các định nghĩa const

Danh sách các định nghĩa const được bổ sung mới trên PS-CPU của S800.

5. 8. 1. main project

iap/App/km_extend_io.c

Tên định nghĩa const	const km_io io_extend[]		
Số I/O mở rộng	Vị trí bit của I/O mở rộng	Kiểu port của I/O mở rộng	Vị trí chân của I/O mở rộng
TYPE_Output_Extend0	OUTPUT_EXTEND_IR_P_ON_BIT	IR_P_ON_GPIO_Port	IR_P_ON_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_MC_P_ON_BIT	MC_P_ON_GPIO_Port	MC_P_ON_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_RESET_BIT	_RESET_GPIO_Port	_RESET_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_SB_PWR_EN_BIT	SB_PWR_EN_GPIO_Port	SB_PWR_EN_Pin
TYPE_Int_Sts0	OUTPUT_EXTEND_WAKEUP_BIT	WAKEUP_GPIO_Port	WAKEUP_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_ERP_SENSOR_ON_BIT	ERP_SENSOR_ON_GPIO_Port	ERP_SENSOR_ON_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_RST_USB_BIT	_RST_USB_GPIO_Port	_RST_USB_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_ENG_STATUS1_BIT	_ENG_STATUS1_GPIO_Port	_ENG_STATUS1_Pin
TYPE_Output_Extend0	OUTPUT_EXTEND_ENG_STATUS2_BIT	_ENG_STATUS2_GPIO_Port	_ENG_STATUS2_Pin
TYPE_Input_Extend0	INPUT_EXTEND_5V_MONI_BIT	MONI_5V_GPIO_Port	MONI_5V_Pin
TYPE_Input_Extend0	INPUT_EXTEND_24V11_MONI_BIT	MONI_24V11_GPIO_Port	MONI_24V11_Pin
TYPE_Input_Extend0	INPUT_EXTEND_MC_3_3V_ON_MONI_BIT	MC3_3VON_MONI_GPIO_Port	MC3_3VON_MONI_Pin
TYPE_Input_Extend0	INPUT_EXTEND_POWER_MONITOR_BIT	POWER_MONITOR_GPIO_Port	POWER_MONITOR_Pin
TYPE_Input_Extend0	INPUT_EXTEND_HRESET_REQ_BIT	_HRESET_REQ_GPIO_Port	_HRESET_REQ_Pin
TYPE_Input_Extend0	INPUT_EXTEND_AP_PWR_EN_BIT	AP_PWR_EN_GPIO_Port	AP_PWR_EN_Pin
TYPE_Input_Extend0	INPUT_EXTEND_MC_OE_REQ_BIT	MC_OE_REQ_GPIO_Port	MC_OE_REQ_Pin

5.9. Đặc tả hàm

[5.2. Danh sách hàm](#) Mô tả đặc tả chi tiết của các hàm như đã liệt kê.

5.9.1. main project

5.9.1.1. main

[File]

main/App/entry.c

[Dạng gọi hàm]

```
int32_t main(void)
```

[Tổng quan]

MHàm main

[Tham số]

Không có

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int32_t	0	Có đặt giá trị trả về, nhưng về cơ bản chương trình không bao giờ tr ở về từ main

[Chi tiết xử lý]

- Bật clock cho khối điều khiển nguồn
- Gọi xử lý khởi tạo riêng của board (MX_Init()).
- Khởi động WDT
- Khởi tạo trạng thái nội bộ (internal_sts_init)
- Thực hiện khởi tạo ngắt riêng của KM (km_it_init)
- Khởi động timer (TIM3 chạy với chu kỳ 1 ms)
- Lưu trạng thái I/O vào bộ nhớ (io_extend_memory_write())
- Đặt S800 về trạng thái reset (/RESET = 0)
- Kiểm tra trạng thái xả 24V
 - Xả hoàn tất
 - Bật nguồn cho S800 (s800_power_on())
 - Xả chưa hoàn tất
 - Bật cờ kiểm tra xả 24V
- Xử lý khởi tạo nhận I2C (i2c_recv_first())
- Xử lý tín hiệu MSW_ON (msw_on_proc())
- Xử lý tín hiệu POWER_MONITOR (power_monitor_proc())
- Xử lý xác nhận xả 24V (moni_24v_proc())
- Xử lý tín hiệu MC_P_ON (mc_p_on_proc())
- Xử lý tín hiệu IR_P_ON (ir_p_on_proc())
- Xử lý tín hiệu MC3.3V (mc3_3von_moni_proc())
- Xử lý tín hiệu MC_PWR_EN (mc_pwr_en_proc())
- Xử lý tín hiệu /HRESET_REQ (hreset_req_proc())
- Xử lý tín hiệu /RST_USB (rst_usb_proc())
- Xử lý nhận I2C (i2c_recv_wait())
- Refresh bộ định thời watchdog (BSP_WDT_Refresh())
- Quay lại 11.

5.9.1.2. jump2iap

[File]

main/App/entry.c

[Dạng gọi hàm]

```
void jump2iap(void)
```

[Tổng quan]

Nhảy sang IAP project

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Lưu địa chỉ đích nhảy của IAP vào một con trỏ hàm
* Địa chỉ đích nhảy của IAP là địa chỉ bắt đầu của Flash ROM nơi lưu IAP + 4
2. Lưu stack pointer nằm tại địa chỉ bắt đầu của Flash ROM chứa IAP vào thanh ghi sp
3. Nhảy tới đích đã đặt ở bước 1.

5.9.1.3. Delnit

[File]

main/App/entry.c

[Dạng gọi hàm]

void Delnit(void)

[Tổng quan]

Thực hiện xử lý dừng đồng loạt cho từng driver

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Xử lý dừng timer (TIM3)
2. Gọi xử lý dừng I2C
3. Gọi xử lý dừng RTC

5.9.1.4. change_cmd2array

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

static TYPE_Io_Extend_Num change_cmd2array(uint8_t addr)

[Tổng quan]

Lấy chỉ số của biến nội bộ từ địa chỉ 2nd của I2C

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint8_t	addr	Địa chỉ 2nd trong truyền thông I2C

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
TYPE_Io_Extend_Num	Index	Trả về chỉ số của các biến nội bộ (io_extend_memory[], internal_sts_memory[]). Trong trường hợp lỗi, TYPE_Io_Extend_Max được trả về

[Chi tiết xử lý]

1. Kiểm tra địa chỉ 2nd
 - 1.1. EXTEND_OUTPUT_COMMAND
 - 1.1.1. TYPE_Output_Extend0 được trả về
 - 1.2. EXTEND_INPUT_COMMAND
 - 1.2.1. TYPE_Input_Extend0 được trả về
 - 1.3. INT_STS_COMMAND
 - 1.3.1. TYPE_Int_Sts0 được trả về
 - 1.4. INTERNEAL_STS_COMMAND
 - 1.4.1. TYPE_Internal_Sts0 được trả về
 - 1.5. Ngoài các trường hợp trên
 - 1.5.1. TYPE_Io_Extend_Max được trả về

5.9.1.5. io_extend_read

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
uint32_t io_extend_read(uint8_t addr)
```

[Tổng quan]

Đọc từ các port dùng cho I/O mở rộng

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint8_t	addr	Địa chỉ 2nd trong truyền thông I2C

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
uint32_t	Dữ liệu đọc	Đọc dữ liệu của số I/O mở rộng được chỉ định. Nếu số I/O mở rộng được chỉ định nằm ngoài phạm vi, trả về 0.

[Chi tiết xử lý]

1. Lấy chỉ số của biến nội bộ từ địa chỉ 2nd và kiểm tra lỗi chỉ số
 - 1.1. Nếu số I/O mở rộng TYPE_Io_Extend_Max lớn hơn hoặc bằng
 - 1.1.2. Trả về với dữ liệu đọc được đặt bằng 0
 - 1.2. Ngoài trường hợp 1.1.
 - 1.2.1. Chuyển sang bước 2.
2. Kiểm tra chỉ số
 - 2.1. TYPE_Output_Extend0, TYPE_Input_Extend0, TYPE_Int_Sts0 Trong trường hợp
 - 2.1.1. io_extend Lấy dữ liệu của số I/O mở rộng từ [] và đọc từ port I/O
 - 2.1.2. Trả về dữ liệu đọc được làm giá trị trả về.
 - 2.2. TYPE_Internal_Sts0 Trong trường hợp
 - 2.2.1. io_extend_memory Trả về phần chỉ số
 - 2.3. Ngoài các trường hợp trên
 - 2.3.1. Trả về với dữ liệu đọc được đặt bằng 0

5.9.1.6. io_extend_write

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void io_extend_write(uint8_t addr, uint32_t data)
```

[Tổng quan]

Ghi vào port dùng cho I/O mở rộng

[Tham số]

I/O	Tham số	Tên biến	Nội dung
-----	---------	----------	----------

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

I	uint32_t	addr	Địa chỉ 2nd trong truyền thông I2C
I	uint32_t	data	Dữ liệu cần ghi vào số I/O mở rộng được chỉ định data.

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Lấy chỉ số của biến nội bộ từ địa chỉ 2nd và kiểm tra lỗi chỉ số
 - 1.1. Nếu số I/O mở rộng TYPE_Io_Extend_Max lớn hơn hoặc bằng
 - 1.1.2. Kết thúc xử lý
 - 1.2. Ngoài trường hợp 1.1.
 - 1.2.1. Chuyển sang bước 2.
2. io_extend Lấy dữ liệu của số I/O mở rộng từ [] và lưu tạm vào bộ nhớ (io_extend_memory). Giữ dữ liệu trong []).
3. Cập nhật port I/O khi không thuộc các điều kiện sau.
 - 3.1. Khi thay đổi port ERP_SENSOR, thực hiện xử lý chuyên dụng (erp_sensor_proc())
 - 3.2. MC_P_ON Khi thay đổi port thì không thao tác port I/O tại thời điểm này.
 - 3.3. IR_P_ON Khi thay đổi port thì không thao tác port I/O tại thời điểm này.
 - 3.4. SB_PWR_EN Không cho phép thay đổi port.
4. Thực hiện ghi vào port I/O theo dữ liệu ghi.

5.9.1.7. io_extend_memory_write

[File]

main/App/entry.c

[Dạng gọi hàm]

```
void io_extend_memory_write(void)
```

[Tổng quan]

Lưu giá trị IO mở rộng vào bộ nhớ

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. io_extend Lấy dữ liệu của số I/O mở rộng từ [] và, trong bộ nhớ (io_extend_memory[]), giữ dữ liệu theo định dạng điện văn I2C.

5.9.1.8. internal_sts_init

[File]

main/App/entry.c

[Dạng gọi hàm]

```
void internal_sts_init(void)
```

[Tổng quan]

Khởi tạo các biến cho trạng thái nội bộ.

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. io_extend_memory[TYPE_Internal_Sts0] Khởi tạo bằng 0

5.9.1.9. mc3_3von_moni_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void mc3_3von_moni_proc(void)

[Tổng quan]

Xử lý tín hiệu MC3.3VON_MONI

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Không làm gì

* Tính đến 2017/5/1, không có điều khiển nào sử dụng MC3.3VON_MONI

5.9.1.10. power_monitor_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void power_monitor_proc(uint8_t state)

[Tổng quan]

Xử lý tín hiệu POWER_MONITOR

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint8_t	state	Trạng thái của bên gọi STATE_INT : Gọi từ ngắt STATE_NORMAL : Gọi từ trạng thái bình thường

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra trạng thái

1.1. STATE_INT

1.1.1. Đặt MC_P_ON xuống Low.

1.1.2. Đặt thông tin bộ nhớ nội bộ của MC_P_ON thành Low.

1.1.3. Đặt IR_P_ON xuống Low.

1.1.4. Đặt thông tin bộ nhớ nội bộ của IR_P_ON thành Low.

1.1.5. Kiểm tra xem đây có phải lần khởi động đầu tiên hay không.(g_first_power_on) – Khởi động lần đầu

1.1.5.1. S800 tắt nguồn của (s800_power_off())

1.2. STATE_NORMAL

1.2.1. Trạng thái khởi động lại và POWER_MONITOR ở mức High và bộ định thời kiểm tra xả 24V đã hết hạn

1.2.1.1. Bật nguồn của S800 (s800_power_on())

1.2.2. Ngoài trường hợp 1.2.1.

1.2.2.1. Không làm gì

5.9.1.11. hreset_req_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void hreset_req_proc(void)
```

[Tổng quan]

Xử lý tín hiệu HRESET_REQ

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Tắt nguồn của S800 (s800_power_off())
2. Đặt cờ trạng thái nguồn thành POWER_ON_FLG_START

5.9.1.12. extend_output_init

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void extend_output_init(void)
```

[Tổng quan]

Xử lý khởi tạo port đầu ra

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Khởi tạo port đầu ra mở rộng.
2. Trạng thái ngắt được khởi tạo
3. IR_P_ON Đặt xuống Low
4. MC_P_ON Đặt xuống Low
5. /RESET Đặt xuống Low
6. SB_PWR_EN Đặt xuống Low
7. ERP_SENSOR_ON Đặt xuống Low
8. /RST_SLP2 đặt xuống Low
9. /DISCHG Đặt xuống Low
10. MC_PWR_EN Đặt xuống Low

5.9.1.13. s800_power_off

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void s800_power_off(void)
```

[Tổng quan]

Xử lý tắt nguồn S800

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Nguồn lần đầu ON Cờ khởi động `g_first_power_on` bằng 0: Đặt về trạng thái khởi động lần đầu
2. Reset S800 (/RESET đặt xuống Low).
3. Tắt nguồn của S800 (SB_PWR_EN đặt xuống Low).
4. Đặt trạng thái reboot trong thông tin bộ nhớ nội bộ thành reboot
5. Khởi tạo các port đầu ra
6. Bật cờ kiểm tra xả 24V.

5.9.1.14. `moni_24v_proc`

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void moni_24v_proc(void)

[Tổng quan]

Xử lý kiểm tra xả 24V

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra cờ kiểm tra xả 24V
 - 1.1. STATE_ACTIVE
 - 1.1.1. MONI_24V11 Kiểm tra tín hiệu
 - 1.1.1.1. Low
 - 1.1.1.1.1. Đặt bộ định thời giám sát 24V bằng 500 ms
 - 1.1.1.1.2. Đặt cờ kiểm tra 24V thành STATE_NONACTIVE
 - 1.1.1.2. High
 - 1.1.1.2.1. Không làm gì
 - 1.2. Ngoài trường hợp 1.1.
 - 1.2.1. Không làm gì

5.9.1.15. `s800_power_on`

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void s800_power_on(void)

[Tổng quan]

Xử lý bật nguồn S800

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra tín hiệu MSW_ON
 - 1.1. High
 - 1.1.1. Bật nguồn S800 (đặt SB_PWR_EN lên High).
 - 1.1.2. Giải phóng reset của S800 (đặt /RESET lên High).
 - 1.1.3. Đặt cờ trạng thái khởi động (POWER_ON_FLG_START) vào power_on_flg.
 - 1.2. Low
 - 1.2.1. Không làm gì

5.9.1.16. ap_power_en_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void ap_power_en_proc(void)
```

[Tổng quan]

Xử lý tín hiệu AP_PWR_EN

Được gọi từ handler ngắt khi AP_PWR_EN thay đổi

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Điều khiển xử lý tín hiệu RST_USB (rst_usb_procGọi)
2. Nguồn lần đầu ONCờ khởi độngg_first_power_on bằng 1:Đặt về trạng thái không phải khởi động bật nguồn lần đầu

5.9.1.17. erp_sensor_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

```
void erp_sensor_proc(void)
```

[Tổng quan]

Xử lý tín hiệu ERP_SENSOR_ON

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. AP_PWR_ENKiểm tra tín hiệu và thông tin điện văn
 - 1.1. AP_PWR_ENTrong trường hợp tín hiệu ở mức Low và ERP_SENSOR trong điện văn ở mức Low
 - 1.1.1. ERP_SENSOR_ONLow
 - 1.2. Ngoài trường hợp 1.1
 - 1.2.1. ERP_SENSOR_ONĐặt lên High

5.9.1.18. rst_usb_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void rst_usb_proc(uint8_t state)

[Tổng quan]

Xử lý tín hiệu RST_USB

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint8_t	state	Trạng thái của bên gọi STATE_INT : Gọi từ ngắt STATE_NORMAL : Gọi từ trạng thái bình thường

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra trạng thái bên gọi

1.1. Gọi từ ngắt

1.1.1. Kiểm tra tín hiệu AP_PWR_EN

1.1.1.1. Nếu AP_PWR_EN ở mức Low

1.1.1.1.1. Đặt RST_USB xuống Low

1.1.1.2. Nếu AP_PWR_EN ở mức High

1.1.1.2.1. Không làm gì

1.2. Gọi từ trạng thái bình thường

1.2.1. Kiểm tra AP_PWR_EN và RST_SLP2 trong điện văn

1.2.1.1. AP_PWR_EN ở mức High và trường RST_SLP2 trong khung dữ liệu ở mức High

1.2.1.1.1. Đặt RST_USB lên High

1.2.1.2. Ngoài trường hợp 1.2.1.1.

1.2.1.2.1. Đặt RST_USB xuống Low

5.9.1. 19. mc_p_on_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void mc_p_on_proc(void)

[Tổng quan]

Xử lý tín hiệu MC_P_ON

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Trạng thái khởi động (power_on_flg) kiểm tra

1.1. power_on_flg là POWER_ON_FLG_NORMAL Trong trường hợp

1.1.1. Kiểm tra giá trị được lưu trong bộ nhớ (io_extend_memory) MC_P_ON Kiểm tra giá trị của

1.1.1.1. Nếu là High

1.1.1.1.1. MONI_24V11 Giá trị của và AP_PWR_EN, kiểm tra giá trị của POWER_MONITOR

1.1.1.1.1.1. MONI_24V11 ở mức Low và AP_PWR_EN Khi tín hiệu này ở mức High và POWER_MONITOR ở mức High

1.1.1.1.1.1.1. Đặt MC_P_ON lên High

1.1.1.1.1.2. Trong trường hợp các điều kiện khác với trên

1.1.1.1.1.2.1. Đặt MC_P_ON xuống Low

1.1.1.2. Nếu là Low

1.1.1.2.1. Đặt MC_P_ON xuống Low.

1.2. power_on_flg is POWER_ON_FLG_NORMAL Ngoài các trường hợp trên

1.2.1. AP_PWR_EN và MONI_24V11, kiểm tra giá trị của POWER_MONITOR

1.2.1.1. AP_PWR_EN ở mức High và MONI_24V11 ở mức Low và POWER_MONITOR ở mức High

1.2.1.1.1. Đặt MC_P_ON lên High

1.2.1.1.2. Đặt thông tin bộ nhớ nội bộ của MC_P_ON thành High

1.2.1.1.3. Đặt IR_P_ON lên High

1.2.1.1.4. Đặt thông tin bộ nhớ nội bộ của IR_P_ON thành High

1.2.1.1.5. Đặt cờ điều kiện khởi động (power_on_flg) thành POWER_ON_FLG_NORMAL.

1.2.1.2. Khi áp dụng các điều kiện khác với 1.2.1.1.

1.2.1.2.1. Đặt IR_P_ON xuống Low

1.2.1.2.2. Đặt thông tin bộ nhớ nội bộ của IR_P_ON thành Low.

1.2.1.2.3. Đặt MC_P_ON xuống Low

1.2.1.2.4. Đặt thông tin bộ nhớ nội bộ của MC_P_ON thành Low.

5.9.1.20. ir_p_on_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void ir_p_on_proc(void)

[Tổng quan]

Xử lý tín hiệu IR_P_ON

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra giá trị được lưu trong bộ nhớ (io_extend_memory) IR được lưu trong P_ON Kiểm tra giá trị của

1.1. Nếu là High

1.1.1. AP_PWR_EN và MONI_24V11 và kiểm tra giá trị của POWER_MONITOR

1.1.1.1. AP_PWR_EN ở mức High và MONI_24V11 ở mức Low và khi POWER_MONITOR ở mức High

1.1.1.1.1. Đặt IR_P_ON lên High

1.1.1.2. Khi áp dụng các điều kiện khác với trên

1.1.1.2.1. Đặt IR_P_ON xuống Low

1.2. Nếu là Low

1.2.1. Đặt IR_P_ON xuống Low.

5.9.1.21. mc_oe_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void mc_oe_proc(void)

[Tổng quan]

Xử lý tín hiệu /MC_OE

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Không làm gì

※Tính đến 2017/5/1, không có điều khiển nào sử dụng MC_OE

5. 9. 1. 22. mc_pwr_en_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void mc_pwr_en_proc(void)

[Tổng quan]

MC_PWR_ENXử lý tín hiệu

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. MONI_24V11Kiểm tra giá trị của

1.1. Trong trường hợp Low

1.1.1. MC_PWR_ENĐặt xuống Low

1.2. Trong trường hợp High

1.2.1. MC_PWR_ENĐặt lên High

5. 9. 1. 23. msw_on_proc

[File]

main/App/km_extend_io.c

[Dạng gọi hàm]

void msw_on_proc(uint8_t state)

[Tổng quan]

Xử lý tín hiệu MSW_ON

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint8_t	state	Trạng thái của bên gọi STATE_INT : Gọi từ ngắt STATE_NORMAL : Gọi từ trạng thái bình thường

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra trạng thái

1.1. STATE_INT

1.1.1. MSW_ONKiểm tra tín hiệu

1.1.1.1. High

1.1.1.1.1. Bật nguồn cho S800 (s800_power_on())

1.1.1.2. Low

1.1.1.2.1. Tắt nguồn của S800 (s800_power_off())

1.2. Ngoài trường hợp 1.1.

1.2.1. MSW_ONKiểm tra tín hiệu

1.2.1.1. Low

1.2.1.1.1. Chuyển PS-CPU sang chế độ Sleep (BSP PWR enter_sleepmode())

1.2.1.2. High

1.2.1.2.1. Không làm gì

5.9.1.24. rx_comp

[File]

main/App/km_i2c.c

[Dạng gọi hàm]

static void rx_comp(uint32_t event)

[Tổng quan]

Xử lý đánh giá dữ liệu nhận được của I2C

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint32_t	event	Sự kiện ngắt I2C

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- Kiểm tra xem bit ARM_I2C_EVENT_TRANSFER_DONE có được set trong event hay không
 - ARM_I2C_EVENT_TRANSFER_DONE Nếu bit không được set thì trả về mà không làm gì
- Driver_I2C0.GetStatus() Khi directionCheck
 - direction trong trường hợp bằng 0
 - Đặt cờ yêu cầu đọc (i2c_info.read_comp)
 - Đặt kích thước nhận
 - direction trong trường hợp bằng 1
 - Đặt cờ yêu cầu ghi (i2c_info.write_comp)
 - Đặt kích thước nhận

5.9.1.25. change_state

[File]

main/App/km_i2c.c

[Dạng gọi hàm]

static void inline change_state(int next_state, int* comp, slave_transfer_t transfer, uint8_t* buf, uint16_t size)

[Tổng quan]

Hàm chuyển đổi giữa xử lý nhận và xử lý gửi của I2C

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	Int	next_state	Chỉ định lần truyền tiếp theo là yêu cầu ghi hay yêu cầu đọc.
I	int*	comp	Chỉ định sử dụng cờ nào: cờ đọc hay cờ ghi.
I	slave_transfer_t	transfer	Chỉ định con trỏ hàm dùng cho việc nhận hoặc gửi. ※ Con trỏ hàm được chỉ định chỉ có thể là Driver_I2C0.SlaveTransmit() hoặc cDriver_I2C0.SlaveReceive()
I	uint8_t*	buf	Khi nhận: con trỏ để lưu dữ liệu nhận được. Khi gửi: con trỏ tới buffer dữ liệu cần gửi.
I	uint16_t	size	Kích thước dữ liệu cần gửi/nhận

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Kiểm tra Null cho từng con trỏ (comp, transfer, buf)
 - 1.1. Nếu là Null thì trả về mà không làm gì
2. Vô hiệu hóa ngắt I2C (HAL_NVIC_DisableIRQ(I2C1_IRQn)được gọi).
3. Lưu yêu cầu tiếp theo vào biến quản lý trạng thái I2C (i2c_info.state).
4. Khởi tạo cờ cho các yêu cầu đọc/ghi
5. Gọi con trỏ hàm đã chỉ định
6. Bật lại ngắt I2C (HAL_NVIC_EnableIRQ(I2C1_IRQn)được gọi).

5. 9. 1. 26. i2c_recv_first

[File]

main/App/km_i2c.c

[Dạng gọi hàm]

void i2c_recv_first(void)

[Tổng quan]

Hàm được gọi trước khi bắt đầu truyền thông I2C

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Đặt biến quản lý trạng thái I2C thành yêu cầu ghi.
 - * Truyền thông với S800 luôn bắt đầu bằng một yêu cầu ghi
2. Gọi hàm nhận của I2C.

5.9.1.27. i2c_recv_wait

[File]

main/App/km_i2c.c

[Dạng gọi hàm]

```
void i2c_recv_wait(void)
```

[Tổng quan]

Xử lý chính của truyền thông I2C

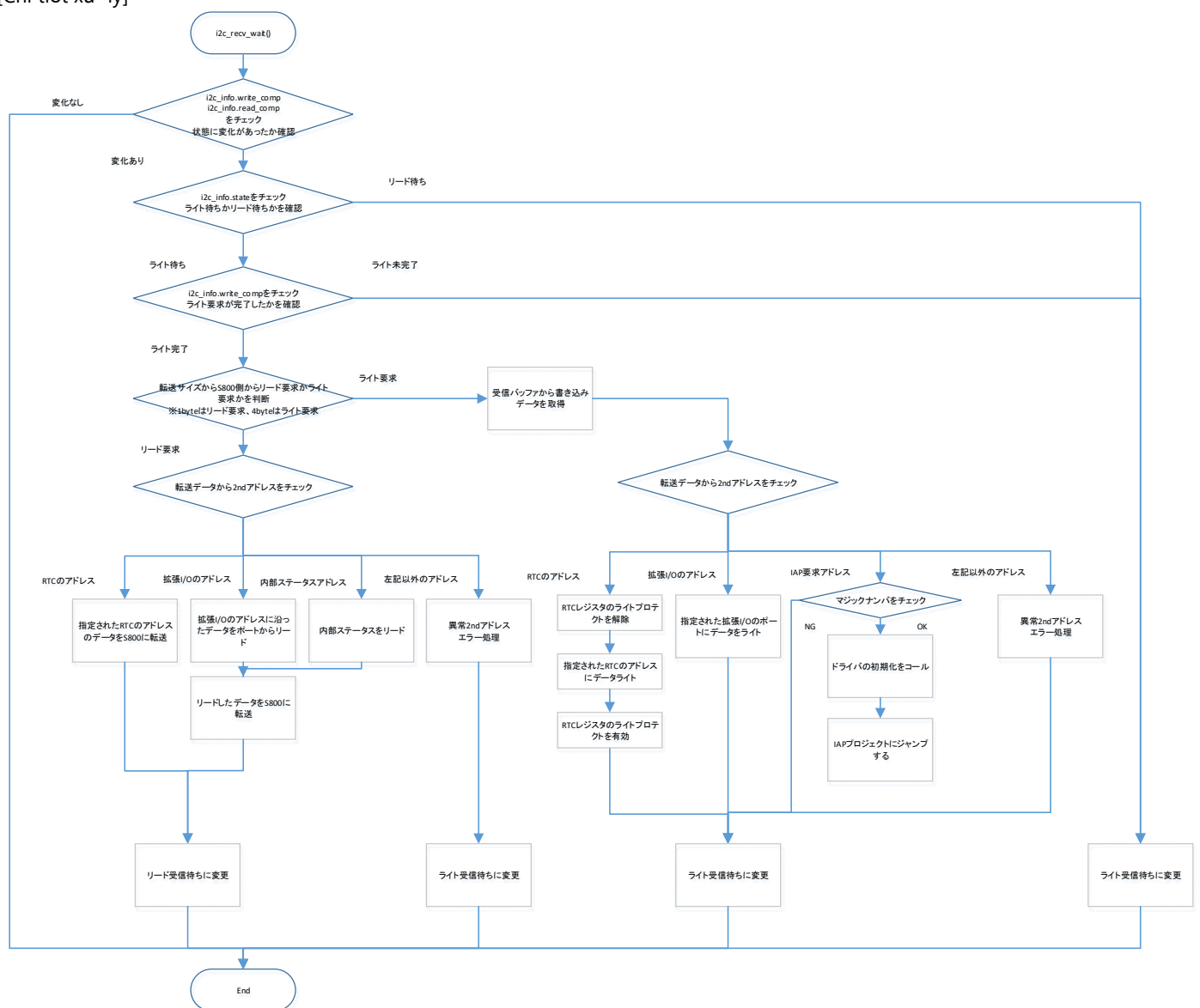
[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]



5.9.1.28. checksum_calc

[File]

main/App/km_i2c.c

[Dạng gọi hàm]

uint16_t checksum_calc(void)

[Tổng quan]

Hàm tính và trả về checksum của chương trình main

[Tham số]

Không có

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int16_t	Checksum	Trả về checksum được tính bằng phép cộng (2Byte)

[Chi tiết xử lý]

- Địa chỉ bắt đầu nơi đặt main project (0x08000000) đến 0x08005000, cộng lần lượt các giá trị
- Lấy 16 bit thấp của giá trị tổng làm checksum
- Trả về checksum(g_checksum_value)

5.9.1.29. hreset_req_proc_handler

[File]

main/App/km_it.c

[Dạng gọi hàm]

static void hreset_req_proc_handler(void)

[Tổng quan]

Handler ngắt cho /HRESET_REQ

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- hreset_flgSTATE_ACTIVE đặt thành

5.9.1.30. km_exti_callback

[File]

main/App/km_it.c

[Dạng gọi hàm]

static void km_exti_callback(GPIO_Pin GPIO_Pin, GPIOEventType event)

[Tổng quan]

Handler ngắt ngoài

* Hàm callback do driver HAL cung cấp

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	PinType	GPIO_Pin	Số chân GPIO (dòng PA)

I	GPIOEventType	event	Loại sự kiện
---	---------------	-------	--------------

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- Kiểm tra loại sự kiện
 - GPIO_EVENT_EXT_INTR thì chuyển sang bước 2.
 - Nếu không phải 1.1. thì kết thúc xử lý
- Kiểm tra số chân
 - /HRESET_REQ Trong trường hợp là chân
 - Thực hiện xử lý cho /HRESET_REQ (hreset_req_proc()Gọi ()).
 - AP_PWR_ENT Trong trường hợp là chân
 - AP_PWR_ENT thực hiện xử lý cho (ap_power_en_proc()Gọi ()).
 - POWER_MONITOR Trong trường hợp là chân
 - POWER_MONITOR thực hiện xử lý cho (power_monitor_proc()Gọi ()).
 - Ngoài các trường hợp trên

5.9.1.31. km_it_init

[File]

main/App/km_it.c

[Dạng gọi hàm]

void km_it_init(void)

[Tổng quan]

Xử lý khởi tạo ngắt tùy chỉnh của KM

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- Gọi xử lý khởi tạo GPIO.
 - ※ Trong quá trình khởi tạo, km_exti_callback đăng ký ()

5.9.1.32. HAL_TIM_PeriodElapsedCallback

[File]

main/App/km_it.c

[Dạng gọi hàm]

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)

[Tổng quan]

Handler ngắt của timer

* Hàm callback do driver HAL cung cấp

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	TIM_HandleTypeDef	htim	Cấu trúc của driver TIMER

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- Kiểm tra NULL cho cấu trúc driver TIMER
 - Nếu NULL thì kết thúc xử lý
- Kiểm tra xem có phải cấu trúc TIM3 hay không

- 2.1. Nếu không phải TIM3 thì kết thúc xử lý
3. Trạng thái đã đăng ký trong biến km_timer là KM_TIMER_START Đếm ngược timer ở trạng thái đó
4. Khi timer hết hạn, đặt trạng thái thành KM_TIMER_END.

5.9.1.33. km_timer_set

[File]

main/App/km_it.c

[Dạng gọi hàm]

int32_t km_timer_set(KM_TIMER_KIND timer_no, MillsecType timer_val, KM_TIMER_STATE timer_flg)

[Tổng quan]

Hàm đăng ký timer

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	KM_TIMER_KIND	timer_no	Số hiệu timer
I	MillsecType	timer_val	Mili giây (MAX=32768ms)
I	KM_TIMER_STATE	timer_flg	Cờ timer KM_TIMER_NORMAL KM_TIMER_START KM_TIMER_END

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int32_t	HAL_OK HAL_ERROR	Nếu đăng ký hoàn tất, HAL_OK Nếu đăng ký thất bại, HAL_ERROR

[Chi tiết xử lý]

1. Kiểm tra lỗi số hiệu timer
 - 1.1. KM_TIMER_KIND_MAX Nếu lớn hơn hoặc bằng giá trị này, trả về đăng ký chưa hoàn tất
2. Kiểm tra lỗi giá trị timer
 - 2.1. KM_TIMER_MAX_TIMER Nếu lớn hơn hoặc bằng giá trị này, trả về đăng ký chưa hoàn tất
3. Kiểm tra lỗi cờ timer
 - 3.1. KM_TIMER_MAX_TIMER Nếu lớn hơn hoặc bằng giá trị này, trả về đăng ký chưa hoàn tất
4. km_timer Đặt trạng thái và giá trị timer vào biến
5. Trả về đăng ký hoàn tất

5.9.1.34. km_timer_get_state

[File]

main/App/km_it.c

[Dạng gọi hàm]

KM_TIMER_STATE km_timer_get_state(KM_TIMER_KIND timer_no)

[Tổng quan]

Hàm lấy trạng thái timer

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	KM_TIMER_KIND	timer_no	Số hiệu timer

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
KM_TIMER_STATE	KM_TIMER_NORMAL KM_TIMER_START KM_TIMER_END	Trạng thái timer

[Chi tiết xử lý]

1. Kiểm tra lỗi số hiệu timer
 - 1.1. KM_TIMER_KIND_MAXTrong trường hợp trênKM_TIMER_NORMALđược trả về
2. Trạng thái củakm_timer được chỉ định sẽ được trả về

5. 9. 1. 35. adc_moni_5v_start_proc

[File]

main/App/km_adc.c

[Dạng gọi hàm]

```
void adc_moni_5v_start_proc(void)
```

[Tổng quan]

Xử lý phát hiện 5V

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. MONI_5Vbắt đầu ADC cho
2. MONI_5Vlấy giá trị ADC của
3. MONI_5Vdừng ADC cho
4. MONI_5VNếu giá trị ADC của tín hiệu này từ 4.75 trở xuống thì chuyển sang 1., nếu từ 4.75 trở lên thì kết thúc xử lý

5. 9. 1. 36. adc_moni_5v_off_proc

[File]

main/App/km_adc.c

[Dạng gọi hàm]

```
void adc_moni_5v_off_proc(void)
```

[Tổng quan]

Xử lý phát hiện sụt 5V

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. MONI_5Vbắt đầu ADC cho
2. MONI_5Vlấy giá trị ADC của
3. MONI_5Vdừng ADC cho
4. MONI_5VNếu giá trị ADC từ 4.45V trở xuống trong 10 lần liên tiếp thì thực hiện điều khiển tắt nguồn S800; nếu từ 4.45 trở lên thì kết thúc xử lý
*Sau khi đã ở mức 4.45V trở xuống 10 lần liên tiếp, nếu trở lại 4.75V trở lên thì thực hiện điều khiển bật nguồn tr ở lại

5. 9. 1. 37. BSP_PWR_enter_stopmode

[File]

common/Drivers/BSP/Src/BSP_PWR_STM32F03x_Nucleo.c

[Dạng gọi hàm]

```
void BSP_PWR_enter_stopmode(void)
```

[Tổng quan]

Hàm chuyển sang chế độ Stop

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Gọi hàm chuyển sang chế độ STOP của driver HAL (HAL_PWR_EnterSTOPMode())

5.9.1.38. BSP_PWR_enter_sleepmode

[File]

common/Drivers/BSP/Src/BSP_PWR_STM32F03x_Nucleo.c

[Dạng gọi hàm]

```
void BSP_PWR_enter_sleepmode(void)
```

[Tổng quan]

Hàm chuyển sang chế độ Sleep

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Gọi hàm chuyển sang chế độ Sleep của driver HAL (HAL_PWR_EnterSLEEPMode)

5.9.2. IAP project**5.9.2.1. main**

[File]

iap/App/entry_iap.c

[Dạng gọi hàm]

```
int32_t main(void)
```

[Tổng quan]

Hàm main

[Tham số]

Không có

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int32_t	0	Có đặt giá trị trả về, nhưng về cơ bản chương trình không bao giờ tr ở về từ main

[Chi tiết xử lý]

1. Gọi xử lý khởi tạo riêng của board (MX_Init()).
2. Chuyển bảng vector sang SRAM

※Chuyển đổi (port) mã mẫu do Hakuto cung cấp

[¥¥10.128.16.110¥data1¥29P.System Integration SoC Development¥03_BSP¥07.Outsourcing Management¥Hakuto \(PS-CPU\)¥From Hakuto¥20160614_Question List, IAP Sample \(STM32F030R8\)](#)

3. Đăng ký ngắt (km_it_init_iap())
4. Refresh bộ định thời watchdog (BSP_WDT_Refresh())
5. Thực hiện xử lý khởi tạo I2C (i2c_recv_firstgọi ())
6. Xử lý nhận I2C (i2c_recv_wait())
7. Refresh bộ định thời watchdog (BSP_WDT_Refresh())
8. Quay lại 5.

5.9.2.2. rx_comp

Vì hàm cùng tên trong main project có nội dung xử lý hoàn toàn giống nhau, [5.9.1.24.rx_comp](#) để tham khảo

5.9.2.3. change_state

Vì hàm cùng tên trong main project có nội dung xử lý hoàn toàn giống nhau, [5.9.1.25.change_state](#) để tham khảo

5.9.2.4. i2c_recv_first

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

```
void i2c_recv_first(void)
```

[Tổng quan]

Hàm được gọi trước khi bắt đầu truyền thông I2C

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Đặt biến quản lý trạng thái I2C thành yêu cầu ghi.
* Truyền thông với S800 luôn bắt đầu bằng một yêu cầu ghi
2. Gọi hàm nhận của I2C.
3. Thực hiện xử lý khởi tạo driver flash (Gọi Driver_Flash0.Initialize (NULL))
4. Xóa vùng flash ROM của main project

5.9.2.5. i2c_recv_wait

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

```
void i2c_recv_wait(void)
```

[Tổng quan]

Xử lý chính của truyền thông I2C

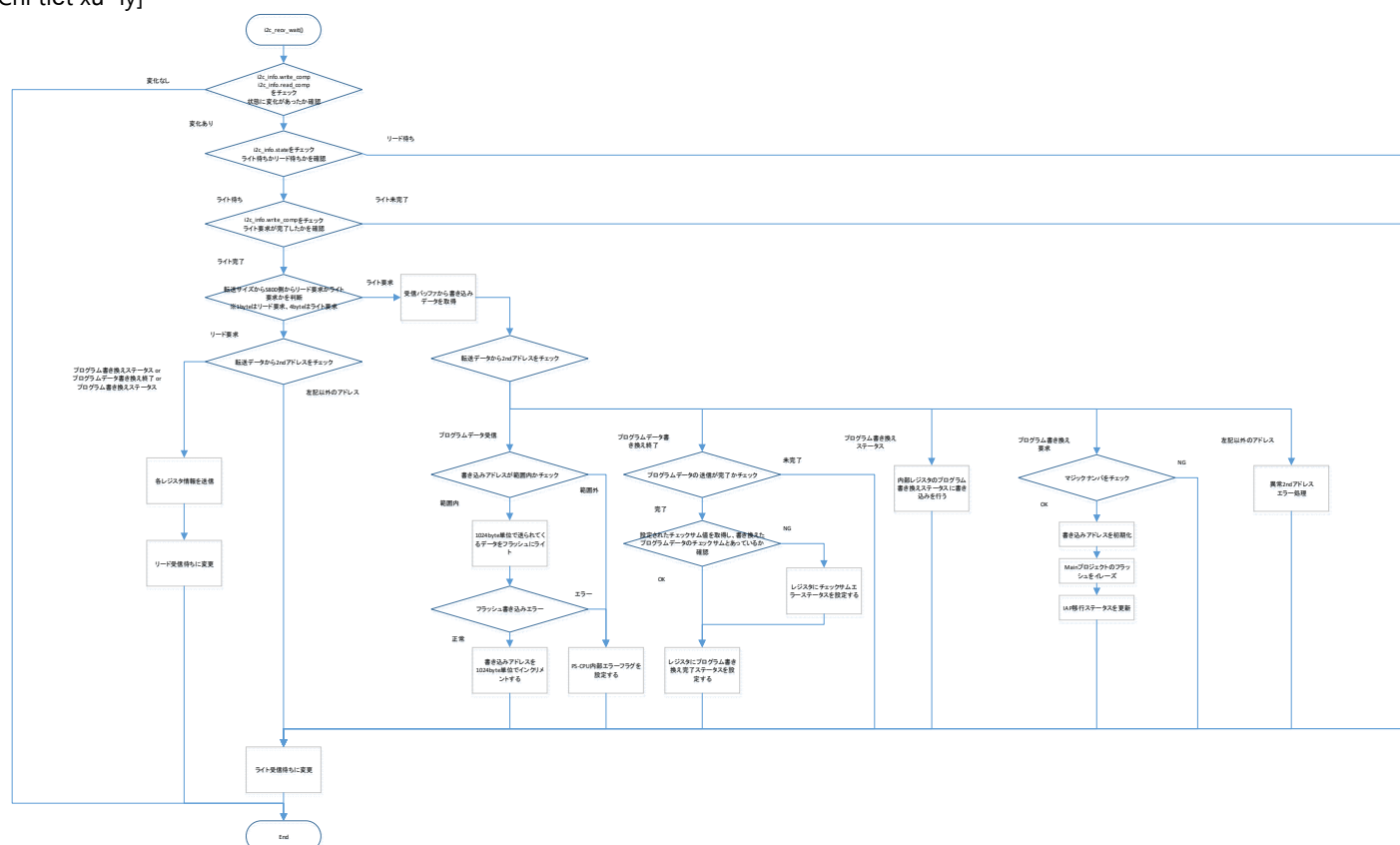
[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]



5.9.2.6. flash write 32

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

```
static int32_t flash_write_32(uint32_t addr, void *data)
```

[Tổng quan]

Ghi vào flash ROM theo đơn vị 32 bit

[Tham số]

I/O	Tham số	Tên biến	Nội dung
-----	---------	----------	----------

BSP cho S800 PS-CPU Đặc tả điều khiển chi tiết

I	uint32_t	addr	Địa chỉ flash
I	void*	data	Con trỏ tới dữ liệu

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int32_t	ARM_DRIVER_OK ARM_DRIVER_ERROR	Ghi OK (giá trị trả về của driver được gọi bên trong) Ghi NG (giá trị trả về của driver được gọi bên trong, hoặc địa chỉ không hợp lệ, hoặc con trỏ dữ liệu là NULL)

[Chi tiết xử lý]

1. Kiểm tra phạm vi địa chỉ
 - 1.1. MAIN_APPLICATION_ADDRESSNhỏ hơn và,MAIN_APPLICATION_ADDRESS + MAIN_APPLICATION_SIZElớn hơn hoặc bằng
 - 1.1.1. Trả về lỗi
 - 1.2. Ngoài các trường hợp trên
 - 1.2.1. Chuyển sang bước 2.
2. Kiểm tra NULL cho con trỏ dữ liệu
 - 2.1. Nếu data là NULL
 - 2.1.1. Trả về lỗi
 - 2.2. Ngoài các trường hợp trên
 - 2.2.1. Chuyển sang 3.
3. Ghi vào flash (Driver_Flash0.ProgramDataGọi)

5.9.2.7. main_project_flash_erase

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

static void main_project_flash_erase(void)
--

[Tổng quan]

Hàm xóa flash của chương trình main

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. MAIN_APPLICATION_ADDRESStừMAIN_APPLICATION_SIZEXóa theo sector cho đến

5.9.2.8. check_sum_calc

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

static int32_t check_sum_calc(uint32_t data)
--

[Tổng quan]

Hàm kiểm tra checksum dữ liệu chương trình main

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	uint32_t	data	Dữ liệu checksum được gửi từ phía master (Giá trị của thanh ghi hoàn tất ghi lại dữ liệu chương trình)

[Giá trị trả về]

Kiểu	Giá trị trả về	Nội dung
int32_t	ARM_DRIVER_OK	Checksum khớp
	ARM_DRIVER_ERROR	Checksum không khớp

[Chi tiết xử lý]

- Lấy giá trị checksum từ dữ liệu được truyền vào làm tham số, theo định dạng của thanh ghi hoàn tất ghi lại dữ liệu chương trình
- Cộng dồn từng byte dữ liệu trong vùng chương trình main (bỏ qua tràn số)
- Kiểm tra các giá trị checksum ở bước 1. và 2.
 - Khớp
 - ARM_DRIVER_OK được trả về
 - Không khớp
 - ARM_DRIVER_ERROR được trả về

5.9.2.9. msw_on_proc_iap

[File]

iap/App/km_extend_io_iap.c

[Dạng gọi hàm]

void msw_on_proc_iap(void)

[Tổng quan]

Hàm xử lý tín hiệu công tắc nguồn chính (nguồn sơ cấp)

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- MSW_ON=0 (Công tắc chính OFF) Kiểm tra trạng thái
- Hoàn tất ghi lại dữ liệu chương trình (0x94) Xác nhận rằng thanh ghi đã được set
- MSW_ON=0 Chỉ trong trường hợp này mới thực hiện software reset HAL_NVIC_SystemReset() Thực thi hàm

5.9.2.10. km_exti_callback_iap

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

void km_exti_callback_iap(PinType GPIO_Pin, GPIOEventType event)
--

[Tổng quan]

Handler ngắt ngoài để nhận ngắt của công tắc nguồn chính

[Tham số]

I/O	Tham số	Tên biến	Nội dung
I	PinType	GPIO_Pin	Số chân GPIO
I	GPIOEventType	event	Loại sự kiện

[Giá trị trả về]

Không có

[Chi tiết xử lý]

- Kiểm tra sự kiện ngắt
- MSW_ON Trong trường hợp có ngắt, thực hiện xử lý ngắt của công tắc nguồn chính msw_on_proc_iap()

5.9.2.11. km_it_init_iap

[File]

iap/App/km_i2c_iap.c

[Dạng gọi hàm]

void km_it_init_iap(void)

[Tổng quan]

Xử lý khởi tạo ngắt

[Tham số]

Không có

[Giá trị trả về]

Không có

[Chi tiết xử lý]

1. Handler ngắt km_exti_callback_iap Đăng ký ()

6. Kiểm chứng

6.1. Môi trường thực thi

Thiết bị và phần mềm cần thiết

- PC (đã cài Keil)
- Board MFP hoặc board đánh giá PS-CPU, môi trường ảo
- Máy hiện sóng, v.v.

6.2. Các quan điểm kiểm chứng

Dưới đây trình bày các quan điểm kiểm chứng của PS-CPU.

6.2.1. Thực hiện kiểm chứng UT

- a. Đối với mã nguồn được tạo mới IC0/C1 độ bao phủ (coverage) 100% và thực hiện.

6.2.2. Khởi động

- a. main project khởi động được.
- b. iap project khởi động được.

6.2.3. Kiểm tra các loại ngắt.

- a. Ngắt xảy ra và handler được gọi.
- b. Ngắt timer hoạt động đúng với chu kỳ 1 ms.
- c. Các ngắt gửi/nhận I2C được thực hiện đúng.

6.2.4. Truyền thông I2C

- a. Việc giao tiếp với thiết bị master được thực hiện đúng theo định dạng.
- b. Có thể lấy được giá trị checksum.
- c. Có thể lấy được thông tin phiên bản.

6.2.5. Điều khiển port

- a. Các giá trị đầu vào của port đầu vào có thể được tham chiếu chính xác.
- b. Các giá trị xuất ra từ port đầu ra là chính xác.
- c. Các port được điều khiển đúng theo các điều kiện đầu vào/đầu ra.

6.2.6. IAP

- a. Dữ liệu chương trình được truyền từ master có thể được ghi lại chính xác.
- b. Chương trình đã ghi lại hoạt động khi công tắc nguồn chuyển từ OFF $\Rightarrow$ ON.

6.2.7. Xử lý mất điện tức thời

- a. Trước khi bật công tắc nguồn, xác nhận rằng PowerMonitor bị ngắt và trình tự tắt nguồn được thực hiện.
- b. Công tắc nguồn ON Sau đó, xác nhận rằng khi PowerMonitor bị ngắt và HRESET_REQ đến thì trình tự tắt nguồn được thực hiện.